

INDEX

[illegible]

[illegible]

Ex.No.1

INSTALLATION OF WINDOWS OPERATING SYSTEM

DATE:

AIM :

To write a procedure to install Windows Operating System

Installing Windows7 Operating System:

System Requirements

If you are currently running Windows Vista, the good news is that you are ready for Windows 7. Tests performed by various parties have consistently confirmed that Windows 7 outperformed Windows Vista on a similar hardware configuration.

If you are coming from previous versions of Windows (pre-Vista), take note of the following *suggested* hardware requirements:

- 1 GHz or faster 32-bit or 64-bit processor
- 1 GB RAM (for 32-bit) or 2 GB RAM (for 64-bit)
- 16 GB of available disk space (for 32-bit) or 20 GB of available disk space (for 64-bit)
- DirectX 9 graphics device with Windows Display Driver Model 1.0 or higher (for Aero—the graphical user interface and default theme in most editions of Windows 7)

Installing Windows 7

If you are currently running Windows Vista (with Service Pack 1), you can upgrade to Windows 7 directly from within Vista. Windows XP users will need to install a fresh copy of Windows 7.

The following steps will walk you through the process of installing Windows 7 on a fresh computer.

Installing Windows 7 is straightforward—if you're doing a clean install, simply boot up your computer with the Windows 7 installation DVD inside the DVD drive and instruct your computer to boot from the DVD (you may need to press a key, such as F11 or F12, while the computer is starting to enter the boot selection screen). If you're upgrading, simply boot into Windows Vista, insert the disc, and run the installer (if you are using Windows XP, see the previous sidebar).

When the installer has booted up, you will be greeted with the screen shown in [Figure 1](#) (the upgrade screen is slightly different; you will have an option to check the compatibility of your system or start the installation). You will be asked to select the language to install, the time and currency format, and your keyboard type



Fig 1:Installing Windows 7: the first step

With the selections made, you can now install Windows 7 by clicking the “Install now” button (see [Figure 1-3](#)).

You will be asked to accept the license agreement. (If you are upgrading, you’ll first have the option to go online to get any updates to the installer first.) Check the licensing checkbox and continue.

On the next screen, you have a choice between upgrading your existing Windows or installing a fresh copy of Windows. If you are using Windows XP or earlier, the first option will not work for you—select the Custom (advanced) option

On the next screen, you have a choice between upgrading your existing Windows or installing a fresh copy of Windows. If you are using Windows XP or earlier, the first option will not work for you—select the Custom (advanced) option

For a fresh installation, you will be asked to select a disk for installing Windows 7. Select the appropriate disk and click Next (see Figure 3). If you are upgrading, the Windows 7 installer will generate a compatibility report and save it to your desktop.



Fig.2 .Click the “Install now” button to start the Windows 7 installation process

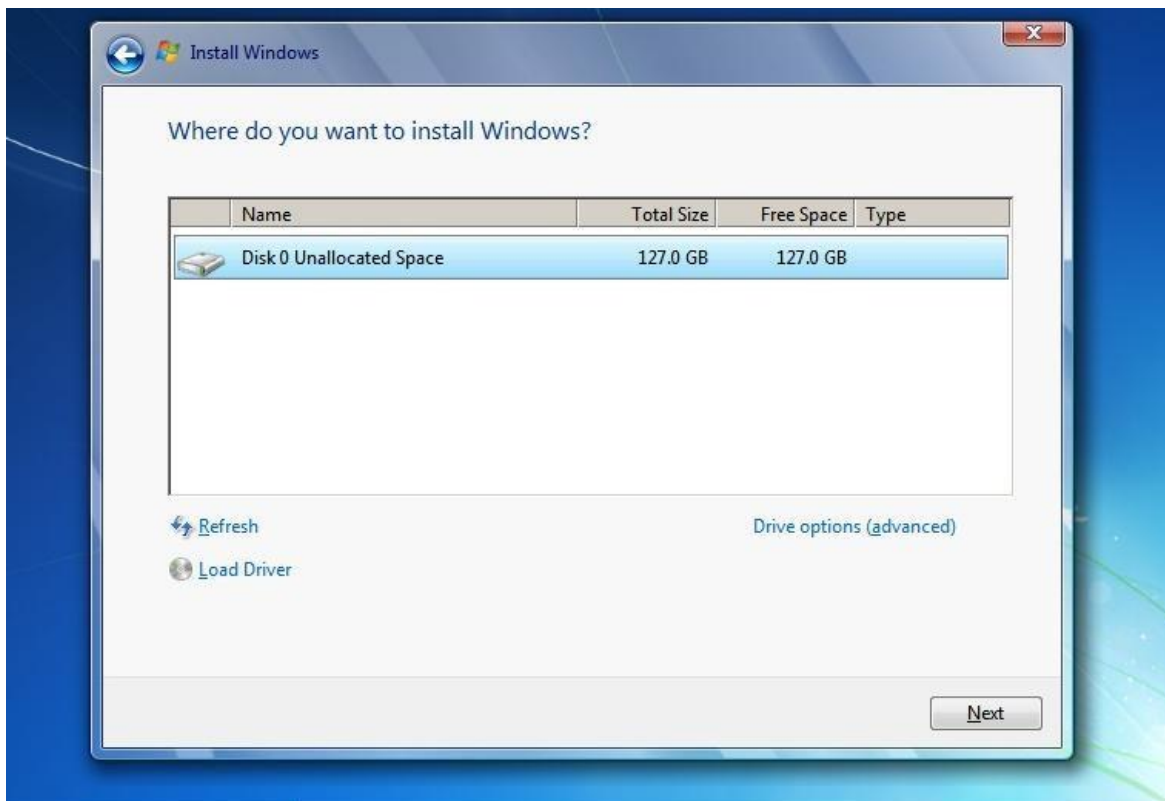


Fig.3. Selecting the disk for installing Windows 7

Windows will now take some time to copy all the files into the selected disk and proceed with the installation (see [Figure 4](#)). This will take about 20–30 minutes, depending on the speed of your computer.

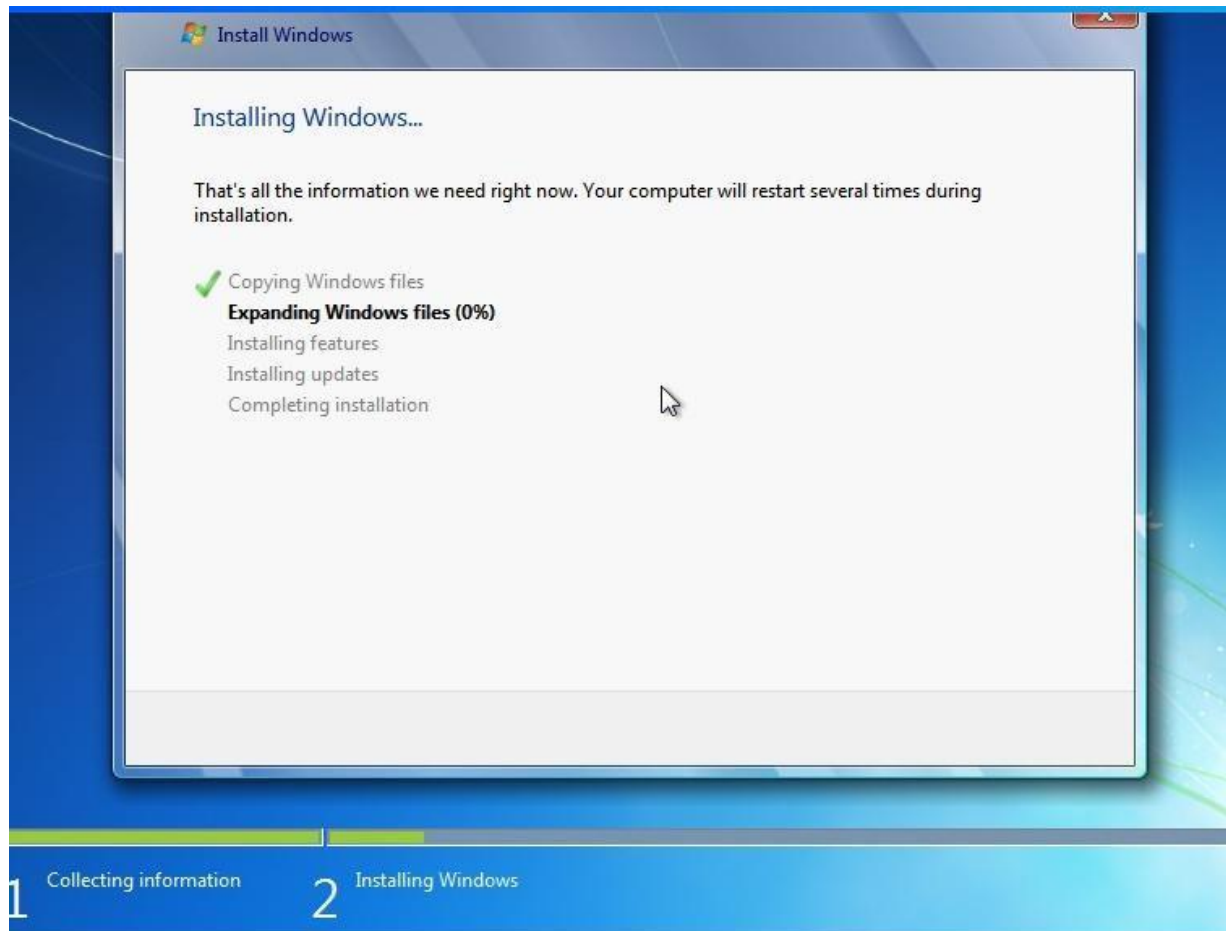


Fig 4 . Windows 7 proceeding with the installation

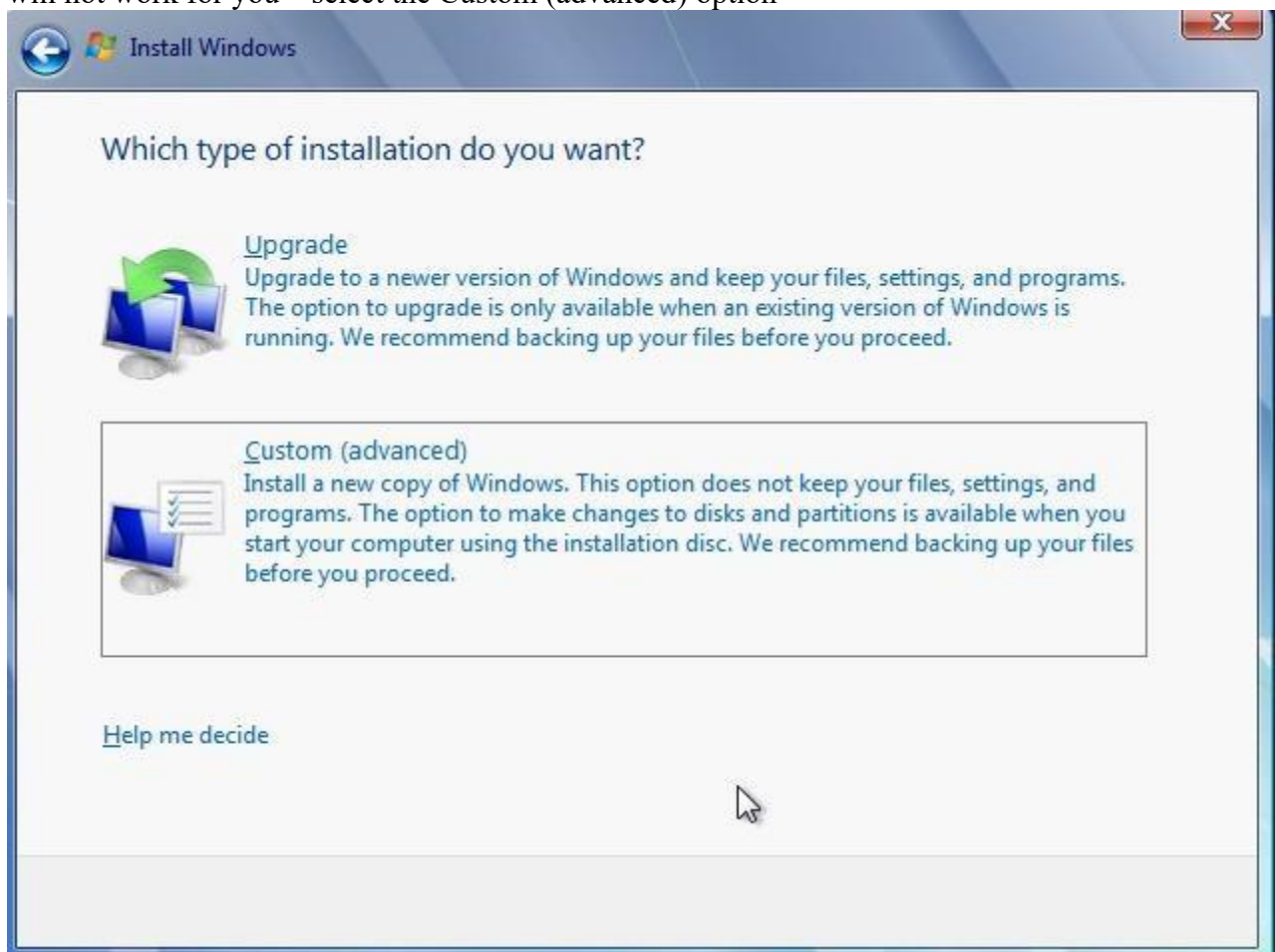
When the installation is complete, Windows 7 will restart. After Windows 7 has been restarted, you should see the screen shown in [Figure 1-7](#). Provide a username; your computer name will be created based on what you have entered (you can change it to another name if you want to after the installation). Click Next.



Fig 5 .Click the “Install now” button to start the Windows 7 installation process

On the next screen, you have a choice between upgrading your existing Windows or installing a fresh copy of Windows. If you are using Windows XP or earlier, the first option

will not work for you—select the Custom (advanced) option



RESULT:

Thus the procedure to install Windows Operating System is done successfully.

EX.NO .2

DATE :

ILLUSTRATION OF BASIC UNIX COMMANDS

AIM :

To study and execute the basic commands in unix.

COMMAND:

1. Date Command:

This command is used to display the current data and time.

Syntax :

\$date -options

Options : -

- a = Abbreviated weekday.
- A = Full weekday.
- b = Abbreviated month.
- B = Full month.
- c = Current day and time.
- C = Display the century as a decimal number.
- d = Day of the month.
- D = Day in „mm/dd/yy“ format
- h = Abbreviated month day.
- H = Display the hour.
- L = Day of the year.
- m = Month of the year.
- M = Minute.
- P = Display AM or PM
- S = Seconds
- T = HH:MM:SS format
- u = Week of the year.
- y = Display the year in 2 digit.
- Y = Display the full year.
- Z = Time zone .

To change the format :

Syntax :

\$date “+%H-%M-%S”

Output:

16-24-19

2. Calendar Command:

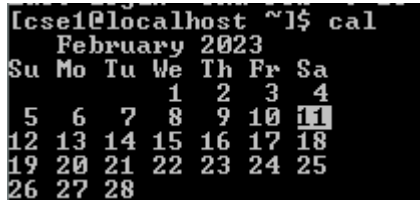
This command is used to display the calendar of the year or the particular month of calendar year.

Syntax :

a.\$cal <year>

b.\$cal <month><year>

Here the first syntax gives the entire calendar for given year & the second Syntax gives the calendar of reserved month of that year.

Output:

```
[cse1@localhost ~]$ cal
February 2023
Su Mo Tu We Th Fr Sa
   1  2  3  4
  5  6  7  8  9 10 11
 12 13 14 15 16 17 18
 19 20 21 22 23 24 25
 26 27 28
```

3. Echo Command:

This command is used to print the arguments on the screen.

Syntax : \$echo <text>

Multi line echo command:

To have the output in the same line, the following commands can be used.

Syntax : \$echo text

To have the output in different line, the following command can be used.

Syntax : \$echo "text

>line2

>line3"

Input &Output:

```
[cse35@localhost ~]$ echo "Hello
```

```
> how
```

```
> r
```

```
> u"
```

```
Hello
```

```
how
```

```
r
```

```
u
```

4.'who' Command :

It is used to display who are the users connected to our computer currently.

Syntax : \$who -option's

Options : -

H-Display the output with headers.

b–Display the last booting date or time or when the system was lastely rebooted.

Input &Output:

```
[cse35@localhost ~]$ who
cse35 pts/1 2017-12-28 16:06 (172.16.10.123)

[cse35@localhost ~]$ who -H
NAME LINE TIME COMMENT
cse35 pts/1 2017-12-28 16:06 (172.16.10.123)

[cse35@localhost ~]$ who -b
system boot 2017-12-28 16:03
```

5.'who am i' Command :

Display the details of the current working directory.

Syntax : \$who am i

Input &Output:

```
[cse35@localhost ~]$ who am i
cse35 pts/1 2017-12-28 16:06 (172.16.10.123)
```

6.'tty' Command :

It will display the terminal name.

Syntax : \$tty

```
[cse1@localhost ~]$ tty
/dev/pts/1
```

7.'Binary' Calculator Command:

Use *bc* is to enter the calculator's own shell. We can run many calculations in a row.

\$ bc

bc 1.06.95

Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006 Free Software Foundation, Inc.

This is free software with ABSOLUTELY NO WARRANTY.

For details type 'warranty'.

Now we can type in calculations or commands, one per line:

1+1

OUTPUT:

2

Input2:

(4+7)*2

4+7*2

8.'CLEAR' Command:

It is used to clear the screen.

Syntax : `$clear`

9.'MAN' Command:

It helps us to know about the particular command and its options & working. It is like "help command in windows.

Syntax : `$man <command name>`

10. tput Command:

The **tput** command uses the **terminfo** database to make terminal-dependent information available to the shell. The **tput** command outputs a string if the attribute *CapabilityName* is of type *string*. The output string is an integer if the attribute is of type *integer*. If the attribute is of type *Boolean*, the **tput** command sets the exit value (0 for TRUE, 1 for FALSE), and produces no other output.

Syntax : `$tput <argument>`

11. LIST Command:

It is used to list all the contents in the current working directory.

Syntax: `$ ls -options <arguments>`

If the command does not contain any argument means it is working in the Current directory.

Options:

- a- used to list all the files including the hidden files.
- c- list all the files columnwise.
- d- list all the directories.
- m- list the files separated by commas.
- p- list files include "/" to all the directories.
- r- list the files in reverse alphabetical order.
- f- list the files based on the list modification date.
- x-list in column wise sorted order.

Input &Output:

```
[cse35@localhost ~]$ ls
```

```
aaa ff fff
```

```
[cse35@localhost ~]$ ls -a
```

```
. aaa .bash_logout .bashrc ff .gnome2
```

```
.. .bash_history .bash_profile .emacs fff .zshrc
```

```
[cse35@localhost ~]$ ls -c
```

```

ff fff aaa

[cse35@localhost ~]$ ls -d
.

[cse35@localhost ~]$ ls -m
aaa, ff, fff

[cse35@localhost ~]$ ls -p
aaa/ ff fff/

[cse35@localhost ~]$ ls -r
fff ff aaa

[cse35@localhost ~]$ ls -f
.emacs .bash_profile ff .bash_logout fff .gnome2
.bashrc .. . aaa .bash_history .zshrc

[cse35@localhost ~]$ ls -x
aaa ff fff

[cse35@localhost ~]$

```

DIRECTORY RELATED COMMANDS:

1. Present Working Directory Command :

To print the complete path of the current working directory.

Syntax : \$pwd

Input &Output:

```
[cse35@localhost ~]$ pwd
```

```
/home/cse35
```

2. MKDIR Command :

To create or make a new directory in a current directory.

Syntax : \$mkdir <directory name>

Input &Output:

```
[cse35@localhost ~]$ mkdir folder1
```

3. CD Command :

To change or move the directory to the mentioned directory.

Syntax : \$cd <directory name>.

Output:

```
[cse1@localhost ~]$ cd aaa
[cse1@localhost aaa]$ cd
[cse1@localhost ~]$
```

4. RMDIR Command :

To remove a directory in the current directory & not the current directory itself.

Syntax : **\$rmdir <directory name>**

FILE RELATED COMMANDS:

1. CREATE A FILE:

To create a new file in the current directory we use CAT command.

Syntax :

\$cat > filename

The > symbol is redirectory we use cat command.

Type '**Ctrl-d**' at the end to save the file.

2. DISPLAY A FILE:

To display the content of file mentioned we use CAT command without ">" operator.

Syntax :

\$cat filename

Input &Output:

```
[cse35@localhost ~]$
```

```
[cse35@localhost ~]$ cat >example2
```

```
hi
```

```
hello
```

```
this is cat command file
```

```
[cse35@localhost ~]$ cat example2
```

```
hi
```

```
hello
```

```
this is cat command file
```

3. COPYING CONTENTS:

To copy the content of one file with another. If file does not exist, a new file is created and if the file exists with some data then it is overwritten.

Syntax :

\$ cat <filename source>>><destination filename>

\$ cat <source filename>>><destination filename> it is avoid overwriting.

Options : -

-n content of file with numbers included with blank lines.

Input &Output:

```
[cse35@localhost ~]$ cat example2>>example3
```

```
[cse35@localhost ~]$ cat example3
```

```
hi
```

```
hello
```

```
this is cat command file
```

```
[cse35@localhost ~]$ cat example2
```

```
hi
```

```
hello
```

```
this is cat command file
```

4. SORTING A FILE:

To sort the contents in alphabetical order in reverse order.

Syntax :

\$sort <filename >

Input &Output:

```
[cse35@localhost ~]$ sort example2
```

```
hello
```

```
hi
```

```
this is cat command file
```

5. COPYING CONTENTS FROM ONE FILE TO ANOTHER:

To copy the contents from source to destination file. So that both contents are same.

Syntax :

\$cp <source filename><destination filename>

Output:

```
[cse35@localhost ~]$ cat example2
```

```
hi
```

```
hello
```

```
this is cat command file
```



```
[cse35@localhost ~]$ cp example2 example4
```

```
[cse35@localhost ~]$ cat example4
```

```
hi
```

```
hello
```

```
this is cat command file
```

```
[cse35@localhost ~]$
```

6. MOVE Command:

To completely move the contents from source file to destination file and to remove the source file.

Syntax :

```
$ mv <source filename> <destination filename>
```

Output:

```
[cse35@localhost ~]$ cat >yy.txt
```

```
This is my first file  
created for copy content  
thankyou
```

```
[cse35@localhost ~]$ cat yy.txt
```

```
This is my first file  
created for copy content  
thankyou
```

```
[cse35@localhost ~]$ mv yy.txt xx.txt
```

```
[cse35@localhost ~]$ cat yy.txt
```

```
cat: yy.txt: No such file or directory
```

```
[cse35@localhost ~]$ cat xx.txt
```

```
This is my first file  
created for copy content  
thankyou
```

7. REMOVE Command:

Use the **rm command** to remove files you no longer need. The rm command removes the entries for a specified file, group of files, or certain select files from a list within a directory. User confirmation, read permission, and write permission are not required before a file is removed when you use the rm command.

Syntax :

```
$rm <filename>
```

8. WORD Command:

To list the content count of no of lines, words, characters.

Syntax :

\$wc <filename>

Options :

-c – to display no of characters.

-l – to display only the lines.

-w – to display the no of words.

Input &Output:

[cse35@localhost ~]\$ cat xx.txt

This is my first file

created for copy content

thankyou

[cse35@localhost ~]\$ wc xx.txt

3 10 56 xx.txt

9. LINE PRINTER:

submits files for printing or alters a pending job.

Syntax :

\$lp <filename>

10. PAGE Command:

This command is used to display the contents of the file page wise & next page can be viewed by pressing the enter key.

Syntax :

\$pg <filename>

11. FILTERS AND PIPES

HEAD: It is used to display the top ten lines of file.

Syntax: \$head <filename>

TAIL: This command is used to display the last ten lines of file.

Syntax: \$tail <filename>

PAGE: This command shows the page by page a screen full of information is displayed after which the page command displays a prompt and passes for the user to strike the enter key to continue scrolling.

Syntax: \$ls -a\p

MORE: It also displays the file page by page .To continue scrolling with more command, press the space bar key.

Syntax: `$more <filename>`

GREP: The grep filter searches a file for a particular pattern of characters, and displays all lines that contain that pattern. The pattern that is searched in the file is referred to as the regular expression (grep stands for global search for regular expression and print out).

Syntax:

`grep [options] pattern [files]`

Options Description

- c : This prints only a count of the lines that match a pattern
- h : Display the matched lines, but do not display the filenames.
- i : Ignores, case for matching
- l : Displays list of a filenames only.
- n : Display the matched lines and their line numbers.
- v : This prints out all the lines that do not matches the pattern
- e **exp** : Specifies expression with this option. Can use multiple times.
- f **file** : Takes patterns from file, one per line.
- E : Treats pattern as an extended regular expression (ERE)
- w : Match whole word
- o : Print only the matched parts of a matching line, with each such part on a separate output line.

- A **n** : Prints searched line and nlines after the result.
- B **n** : Prints searched line and n line before the result.
- C **n** : Prints searched line and n lines after before the result.

\$cat > geekfile.txt

unix is great os. unix was developed in Bell labs.

learn operating system.

Unix linux which one you choose.

uNix is easy to learn.unix is a multiuser os.Learn unix .unix is a powerful.

\$grep -i "UNix" geekfile.txt

Output:

unix is great os. unix was developed in Bell labs.

Unix linux which one you choose.

uNix is easy to learn.unix is a multiuser os.Learn unix .unix is a powerful.

Case insensitive search : The -i option enables to search for a string case insensitively in the given file. It matches the words like “UNIX”, “Unix”, “unix”.

PIPE: It is a mechanism by which the output of one command can be channeled into the input of another command.

A pipe is a form of redirection (transfer of standard output to some other destination) that is used in Linux and other Unix-like operating systems to send the output of one command/program/process to another command/program/process for further processing.

Syntax: \$who | wc -l

TR: The tr filter is used to translate one set of characters from the standard inputs to another.

Syntax: \$tr "[a-z]" "[A-Z]"

WC: Word Command.Used to list the no of lines,words,characters

Syntax: \$wc <filename>

COMMUNICATION THROUGH UNIX COMMANDS

1. MESG

Mesg controls the access to your terminal by others. It's typically used to allow or disallow other users to write to your terminal (see **write(1)**).

OPTIONS

Tag	Description
y	Allow write access to your terminal.
n	Disallow write access to your terminal.

If no option is given, **mesg** prints out the current access state of your terminal.

Syntax: \$mesg y

Input & Output:

[cse35@localhost ~]\$ mesg

2. Command: WRITE

This command is used to communicate with other users, who are logged in at the same time.

Syntax: \$write <user name>

3. Command: WALL

This command sends message to all users those who are logged in using the unix server.

Syntax: \$wall <message>

4. Command: MAIL

It refers to textual information, which can be transferred from one user to another.

Syntax: \$mail <user name>

5. Command: REPLY

It is used to send reply to specified user.

Syntax: \$reply <user name>

RESULT:

Thus the commands are executed successfully.

Ex.No:2.2

SHELL PROGRAMMING

DATE :

(i) CONCATENATION OF TWO STRINGS

AIM:

To write a shell program to concatenate two strings.

ALGORITHM:

Step1: Enter into the vi editor and go to the insert mode for entering the code

Step2: Read the first string.

Step3: Read the second string

Step4: Concatenate the two strings

Step5: Enter into the escape mode for the execution of the result and verify the output

PROGRAM:

```
echo "enter the first string"
read str1
echo "enter the second string"
read str2
echo "the concatenated string is" $str1$str2
```

Input:

Enter first string: Hello

Enter first string: World

Output:

The concatenated string is HelloWorld

RESULT:

Thus the program is executed successfully.

(ii) COMPARISON OF STRINGS

AIM:

To write a shell program to compare the two strings.

ALGORITHM:

Step1: Enter into the vi editor and go to the insert mode for entering the code

Step2: Read the first string.

Step3: Read the second string

Step4: Compare the two strings using the if loop

Step5: If the condition satisfies then print that two strings are equal else print two strings are not equal.

Step6: Enter into the escape mode for the execution of the result and verify the output

PROGRAM:

```
echo "enter the first string"
read str1
echo "enter the second string"
read str2
if [ $str1 = $str2 ]
then
echo "strings are equal"
else
echo "strings are unequal"
fi
```

OUTPUT:

```
Enter first string: hello
Enter second string: hello
The two strings are equal
```

```
Enter first string: hello
Enter second string: helo
The two strings are not equal
```


RESULT:

Thus the program is executed successfully.

(iii) MAXIMUM OF THREE NUMBERS

AIM:

To write a shell program to find greatest of three numbers.

ALGORITHM:

- Step1: Declare the three variables.
- Step2: Check if A is greater than B and C.
- Step3: If so print A is greater.
- Step4: Else check if B is greater than C.
- Step5: If so print B is greater.
- Step6: Else print C is greater.

PROGRAM:

```
echo "enter A"
read a
echo "enter B"
read b
echo "enter C"
read c
if [ $a -gt $b -a $a -gt $c ]
then
echo "A is greater"
elif [ $b -gt $a -a $b -gt $c ]
then
echo "B is greater"
else
echo "C is greater"
fi
```

OUTPUT:

```
Enter A:23
Enter B:45
Enter C:67
C is greater
```

RESULT:

Thus the program is executed successfully.

(iv) FIBONACCI SERIES

AIM:

To write a shell program to generate Fibonacci series.

ALGORITHM :

Step 1 : Initialize a to 0 and b to 1.

Step 2 : Print the values of 'a' and 'b'.

Step 3 : Add the values of 'a' and 'b'. Store the added value in variable 'c'.

Step 4 : Print the value of 'c'.

Step 5 : Initialize 'a' to 'b' and 'b' to 'c'.

Step 6 : Repeat the steps 3,4,5 till the value of 'a' is less than 10

PROGRAM :

```
clear
echo "How many number of terms to be generated ?"
read n
x=0
y=1
i=2
echo "Fibonacci Series up to $n terms : "
echo "$x"
echo "$y"
while [ $i -lt $n ]
do
    i=`expr $i + 1 `
    z=`expr $x + $y `
    echo "$z"
    x=$y
    y=$z
done
```

OUTPUT:

How many number of terms to be generated?

5

Fibonacci Series up to 5 terms :

0

1

1

2

3

RESULT:

Thus the program is executed successfully.

(v) ARITHMETIC OPERATIONS USING CASE

AIM:

To write a shell program to perform the arithmetic operations using case

ALGORITHM :

- Step 1 : Read the input variables and assign the value
- Step 2 : Print the various arithmetic operations which we are going to perform
- Step 3 : Using the case operator assign the various functions for the arithmetic operators.
- Step 4 : Check the values for all the corresponding operations.
- Step 5 : Print the result and stop the execution.

PROGRAM:

```
echo "Enter Two Numbers"
read a b
echo "What do you want to do? (1 to 5)"
echo "1) Sum"
echo "2) Difference"
echo "3) Product"
echo "4) Quotient"
echo "5) Remainder"
echo "Enter your Choice"
read n
case "$n" in
1) echo "The Sum of $a and $b is `expr $a + $b`";;
2) echo "The Difference between $a and $b is `expr $a - $b`";;
3) echo "The Product of the $a and $b is `expr $a \* $b`";;
4) echo "The Quotient of $a by $b is `expr $a / $b`";;
5) echo "The Remainder of $a by $b is `expr $a % $b`";;
esac
```

OUTPUT:

Enter Two Numbers

3 4

What do you want to do? (1 to 5)

1) Sum

2) Difference

3) Product

4) Quotient

5) Remainder

Enter your Choice

1

The Sum of 3 and 4 is 7

Enter Two Numbers

3 4

What do you want to do? (1 to 5)

1) Sum

2) Difference

3) Product

4) Quotient

5) Remainder

Enter your Choice

2

The Difference between 3 and 4 is -1

RESULT :

Thus the program is executed successfully.

(vi) PROGRAM TO FIND ODD OR EVEN

AIM:

To write a shell program to find Odd or Even

ALGORITHM :

Step 1: Start the Program

Step 1 : Read the input variables and get the number as input number 'n'

Step 2 : Find the mod value of the input number 'n' with 2.

Step 3 : If the remainder is 0 means then the number is even number else the number is odd number

Step 4 : Print the result and stop the execution

PROGRAM:

```
echo -n "Enter number : "  
read n  
rem=$(( $n % 2 ))  
if [ $rem -eq 0 ]  
then  
    echo "$n is even number"  
else  
    echo "$n is odd number"  
fi
```


OUTPUT:

Enter number : 3

3 is odd number

RESULT:

Thus the program is executed successfully.

(vii) PROGRAM TO FIND FACTORIAL OF A NUMBER

AIM:

To write a shell program to find a factorial a number

ALGORITHM :

- Step 1. Get a number
- Step 2. Use for loop or while loop to compute the factorial by using the below formula
- Step 3. $\text{fact}(n) = n * n-1 * n-2 * .. 1$
- Step 4. Display the result.

PROGRAM

```
fact ()
{
    if [ $1 -gt 1 ]; then
        y=`expr $1 - 1`
        fact $y
        x=$(( $1 * $? ))
        return $x
    else
        return 1
    fi
}
echo -e "Enter a number : \c"
read num
fact $num
echo "Factorial of $num is $?."
```

OUTPUT:

Enter a number: 4

Factorial of 4 is 24.

RESULT:

Thus the program is executed successfully.

Ex.No:3.1

PROGRAM TO SIMULATE OPENDIR() AND READDIR()

DATE :

SYSTEM CALL

AIM:

To write a program to simulate Opendir () abd readdir () system call

ALGORITHM:

- Step 1.Start the Program
- Step 2.Create a directory
- Step 3.Open the directory with opendir() System call
- Step 4.If the required directory is not found then display the directory is not found.
- Step 5.Else read the directory with the System call readdir()
- Step 6.Stop the program.

PROGRAM:

```
#include<stdio.h>
#include<dirent.h>
int main()
{
char dirname[10];
DIR*p;
struct dirent *d;
printf("Enter directory name:\n");
scanf("%s",dirname);
p=opendir(dirname);
if(p==NULL)
{
perror("Cannot find directory");
exit(-1);
}
while(d=readdir(p))
printf("%s\n",d->d_name);
return 0;
}
```

OUTPUT:

```
[cse2@localhost]$ mkdir bhu
[cse2@localhost]$ cd bhu
[cse2@localhost bhu]$ cat >first.txt
Hi
Hello
How r u
[cse2@localhost bhu]$ cd
[cse2@localhost]$ cc sample.c
[cse2@localhost]$ ./a.out
Enter directory name:
bhu
..
first.txt
```

RESULT:

Thus the program is executed successfully.

Ex.No:3.2

PROGRAM TO SIMULATE FORK() SYSTEM CALL

DATE :

AIM:

To write a C Program to implement fork() System call.

ALGORITHM:

Step 1:Start the Program

Step 2:Create child process

Step 3:Get the pid value and print the child process id

Step 4:Exit the Program to terminate the current running process using exit() System call.

Step 5:Stop the Program.

PROGRAM:

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    printf("I am: %d\n", (int) getpid());
    pid_t pid = fork();
    printf("fork returned: %d\n", (int) pid);

    if (pid < 0) { /* error occurred */
        perror("Fork failed");
    }
    if (pid == 0) { /* child process */
        printf("I am the child with pid %d\n", (int) getpid());
        printf("Child process is exiting\n");
        exit(0);
    }
    /* parent process */
    printf("I am the parent waiting for the child process to end\n");
    wait(NULL);
    printf("parent process is exiting\n");
    return(0);
}
```

OUTPUT:

```
[cse2@localhost ~]$ ./a.out
I am: 3577
fork returned: 0
I am the child with pid 3578
Child process is exiting
fork returned: 3578
I am the parent waiting for the child process to end
parent process is exiting
```

RESULT:

Thus the program is executed successfully.

Ex.No:3.3

PROGRAM TO SIMULATE WAIT() SYSTEM CALL

DATE :

AIM:

To write a C Program to implement wait() System call.

ALGORITHM:

- If any process has more than one child processes, then after calling wait(), parent process has to be in wait state if no child terminates.
- If only one child process is terminated, then return a wait() returns process ID of the terminated child process.

If more than one child processes are terminated than wait() reap any *arbitrarily child* and return a process ID of that child process.

- When wait() returns they also define **exit status** (which tells our, a process why terminated) via pointer, If status are not **NULL**.

If any process has no child process then wait() returns immediately "-1"

PROGRAM:

// C program to demonstrate working of wait()

```
#include<stdio.h>
#include<stdlib.h>
#include<sys/wait.h>
#include<unistd.h>
int main()
{
    pid_t cpid;
    if (fork()== 0)
        exit(0);      /* terminate child */
    else
        cpid = wait(NULL); /* reaping parent */
    printf("Parent pid = %d\n", getpid());
    printf("Child pid = %d\n", cpid);

    return 0;
}
```


OUTPUT:

Parent pid = 12345678

Child pid = 89546848

RESULT:

Thus the program is executed successfully.

Ex.No:4.1

IMPLEMENTATION OF FCFS SCHEDULING

DATE :

AIM:

To write a C program to implement the CPU scheduling algorithm for FIRST COME FIRST SERVE.

ALGORITHM:

Step 1: Create the number of process.

Step 2: Get the ID and Service time for each process.

Step 3: Initially, Waiting time of first process is zero and Total time for the first process is the starting time of that process.

Step 4: Calculate the Total time and Processing time for the remaining processes.

Step 5: Waiting time of one process is the Total time of the previous process.

Step 6: Total time of process is calculated by adding Waiting time and Service time.

Step 7: Total waiting time is calculated by adding the waiting time for lack process.

Step 8: Total turn around time is calculated by adding all total time of each process.

Step 9: Calculate Average waiting time by dividing the total waiting time by total number of process.

Step10: Calculate Average turn around time by dividing the total time by the number of process.

Step11: Display the result

PROGRAM:

```
#include<stdio.h>
#include<string.h>
int main()
{
    int n,bt[20],wt[20],tat[20],avwt=0,avtat=0,i,j;
    printf("Enter total number of processes(maximum 20):");
    scanf("%d",&n);
    printf("\nEnter Process Burst Time\n");
    for(i=0;i<n;i++)
    {
        printf("P[%d]:",i+1);
        scanf("%d",&bt[i]);
    }
}
```

```

    wt[0]=0; //waiting time for first process is 0
    for(i=1;i<n;i++)
    {
        wt[i]=0;
        for(j=0;j<i;j++)
            wt[i]+=bt[j];
    }
    printf("\nProcess\t\tBurst Time\tWaiting Time\tTurnaround Time");
    for(i=0;i<n;i++)
    {
        tat[i]=bt[i]+wt[i];
        avwt+=wt[i];
        avtat+=tat[i];
        printf("\nP[%d]\t\t%d\t\t%d\t\t%d",i+1,bt[i],wt[i],tat[i]);
    }
    avwt/=i;
    avtat/=i;
    printf("\n\nAverage Waiting Time:%d",avwt);
    printf("\n\nAverage Turnaround Time:%d",avtat);
    return 0;
}

```

OUTPUT:

```
C:\Users\admin\Desktop\Untitled1.exe
Enter total number of processes<maximum 20>:3
Enter Process Burst Time
P[1]:24
P[2]:3
P[3]:3

Process      Burst Time    Waiting Time   Turnaround Time
P[1]         24            0              24
P[2]         3             24             27
P[3]         3             27             30

Average Waiting Time:17
Average Turnaround Time:27
Process returned 0 (0x0)   execution time : 7.661 s
Press any key to continue.
```

RESULT:

Thus the program is executed successfully.

Ex.No:4.2

IMPLEMENTATION OF SJE SCHEDULING

DATE :

AIM:

To write a C program to implement the CPU scheduling algorithm for Shortest job first.

ALGORITHM:

- Step1: Get the number of process.
- Step2: Get the id and service time for each process.
- Step3: Initially the waiting time of first short process as 0 and total time of first short is process the service time of that process.
- Step4: Calculate the total time and waiting time of remaining process.
- Step5: Waiting time of one process is the total time of the previous process.
- Step6: Total time of process is calculated by adding the waiting time and service time of each process.
- Step7: Total waiting time calculated by adding the waiting time of each process.
- Step8: Total turnaround time calculated by adding all total time of each process.
- Step9: calculate average waiting time by dividing the total waiting time by total number of process.
- Step10: Calculate average turnaround time by dividing the total waiting time by total number of process.
- Step11: Display the result

PROGRAM:

```
#include<stdio.h>
#include<string.h>
void main()
{
    int bt[20],p[20],wt[20],tat[20],i,j,n,total=0,pos,temp;
    float avg_wt,avg_tat;
    printf("Enter number of process:");
    scanf("%d",&n);

    printf("\nEnter Burst Time:\n");
    for(i=0;i<n;i++)
    {
        printf("p%d:",i+1);
        scanf("%d",&bt[i]);
        p[i]=i+1;          //contains process number
```

```

    }

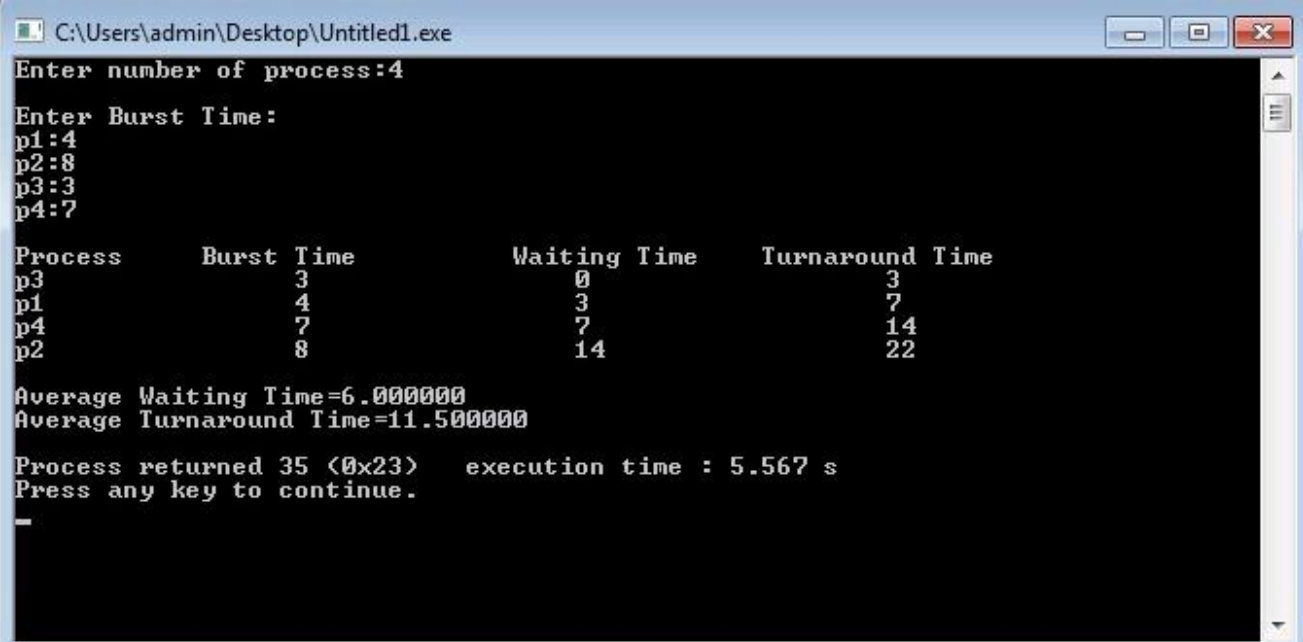
    //sorting burst time in ascending order using selection sort
    for(i=0;i<n;i++)
    {
        pos=i;
        for(j=i+1;j<n;j++)
        {
            if(bt[j]<bt[pos])
                pos=j;
        }
        temp=bt[i];
        bt[i]=bt[pos];
        bt[pos]=temp;

        temp=p[i];
        p[i]=p[pos];
        p[pos]=temp;
    }
    wt[0]=0;    //waiting time for first process will be zero
    for(i=1;i<n;i++)
    {
        wt[i]=0;
        for(j=0;j<i;j++)
            wt[i]+=bt[j];

        total+=wt[i];
    }
    avg_wt=(float)total/n;    //average waiting time
    total=0;
    printf("\nProcess\t Burst Time \tWaiting Time\tTurnaround Time");
    for(i=0;i<n;i++)
    {
        tat[i]=bt[i]+wt[i];    //calculate turnaround time
        total+=tat[i];
        printf("\np%d\t\t %d\t\t %d\t\t%d",p[i],bt[i],wt[i],tat[i]);
    }
    avg_tat=(float)total/n;    //average turnaround time
    printf("\n\nAverage Waiting Time=%f",avg_wt);
    printf("\n\nAverage Turnaround Time=%f\n",avg_tat);
}

```

OUTPUT:



```
C:\Users\admin\Desktop\Untitled1.exe
Enter number of process:4
Enter Burst Time:
p1:4
p2:8
p3:3
p4:7

Process      Burst Time      Waiting Time      Turnaround Time
p3           3              0                3
p1           4              3                7
p4           7              7               14
p2           8             14               22

Average Waiting Time=6.000000
Average Turnaround Time=11.500000

Process returned 35 (0x23)    execution time : 5.567 s
Press any key to continue.
-
```

RESULT:

Thus the program is executed successfully.

Ex.No:4.3

IMPLEMENTATION OF PRIORITY SCHEDULING

DATE :

AIM:

To write a C program to implement CPU scheduling algorithm for priority scheduling.

ALGORITHM:

Step1: Get the number of process, burst time and priority.

Step2: Using for loop $i=0$ to $n-1$ do step 1 to 6.

Step3: If $i=0$, wait time $=0$, $T[0]=b[0]$;

Step4: $T[i]=T[i-1]+b[i]$ and $wt[i]=T[i]-b[i]$.

Step5: Total waiting time is calculated by adding the waiting time for each process.

Step6: Total turn around time is calculated by adding all total time of each process.

Step7: Calculate Average waiting time by dividing the total waiting time by total number of process.

Step8: Calculate Average turn around time by dividing the total time by the number of process.

Step9: Display the result

PROGRAM:

```
#include<stdio.h>
#include<string.h>
int main()
{
    int bt[20],p[20],wt[20],tat[20],pr[20],i,j,n,total=0,pos,temp,avg_wt,avg_tat;
    printf("Enter Total Number of Process:");
    scanf("%d",&n);

    printf("\nEnter Burst Time and Priority\n");
    for(i=0;i<n;i++)
    {
        printf("\nP[%d]\n",i+1);
        printf("Burst Time:");
        scanf("%d",&bt[i]);
        printf("Priority:");
        scanf("%d",&pr[i]);
        p[i]=i+1; //contains process number
    }
```



```

//sorting burst time, priority and process number in ascending order using selection sort
for(i=0;i<n;i++)
{
    pos=i;
    for(j=i+1;j<n;j++)
    {
        if(pr[j]<pr[pos])
            pos=j;
    }
    temp=pr[i];
    pr[i]=pr[pos];
    pr[pos]=temp;
    temp=bt[i];
    bt[i]=bt[pos];
    bt[pos]=temp;
    temp=p[i];
    p[i]=p[pos];
    p[pos]=temp;
}
wt[0]=0; //waiting time for first process is zero
//calculate waiting time
for(i=1;i<n;i++)
{
    wt[i]=0;
    for(j=0;j<i;j++)
        wt[i]+=bt[j];

    total+=wt[i];
}
avg_wt=total/n; //average waiting time
total=0;
printf("\nProcess\t Burst Time \tWaiting Time\tTurnaround Time");
for(i=0;i<n;i++)
{
    tat[i]=bt[i]+wt[i]; //calculate turnaround time
    total+=tat[i];
    printf("\nP[%d]\t\t %d\t\t %d\t\t%d",p[i],bt[i],wt[i],tat[i]);
}
avg_tat=total/n; //average turnaround time
printf("\n\nAverage Waiting Time=%d",avg_wt);
printf("\n\nAverage Turnaround Time=%d\n",avg_tat);
return 0;
}

```

OUTPUT:

```
Telnet 172.16.10.250
Enter Total Number of Process:5
Enter Burst Time and Priority
P[1]
Burst Time:10
Priority:3
P[2]
Burst Time:1
Priority:1
P[3]
Burst Time:2
Priority:4
P[4]
Burst Time:1
Priority:5
P[5]
Burst Time:5
Priority:2
Process Burst Time      Waiting Time      Turnaround Time
P[2]          1          0          1
P[5]          5          1          6
P[1]         10          6         16
P[3]          2         16         18
P[4]          1         18         19
Average Waiting Time=8
Average Turnaround Time=12
[cse50@localhost ~]$ _
```

RESULT:

Thus the program is executed successfully.

Ex.No:4.4

IMPLEMENTATION OF ROUND ROBIN SCHEDULING

DATE :

AIM :

To write a C program to simulate the CPU scheduling algorithm for round robin

ALGORITHM:

Step 1:Start

Step 2:Take input the number of processes in the system (NOP) from the user.

Step 3:For each process, take input its arrival time and burst time from the user and store it in the respective arrays.

Step 4:Take input the time quantum for the process from the user.

Step 5:Set $y = \text{NOP}$ to track the number of processes that are not yet completed.

Step 6:Initialize sum, count, wt and tat to zero.

Step 7:Display the table headers with column names: Process No, Burst Time, TAT, and Waiting Time.

Step 8:While there are processes that are not yet completed ($y > 0$), do the following:

1. Check if the remaining burst time of the current process ($\text{temp}[i]$) is less than or equal to the time quantum and greater than zero. If yes, execute the process for the remaining burst time and add the sum of burst times to the variable sum. Set the remaining burst time of the current process to zero and set count to 1. b.
2. If the remaining burst time of the current process is greater than zero, execute the process for the time quantum and add the time quantum to the variable sum. Reduce the remaining burst time of the current process by the time quantum. c.
3. If the remaining burst time of the current process is zero and count is 1, mark the process as completed, reduce the number of processes that are not yet completed (y), and calculate the waiting time and turnaround time of the completed process. Add the waiting time to the variable wt and the turnaround time to the variable tat. Display the details of the completed process in the table.
4. Check if the current process is the last process in the queue. If yes, set i to zero. Else, check if the arrival time of the next process in the queue is less than or equal to the

current sum of burst times. If yes, set i to the index of the next process. Else, set i to zero.

Step 9: Calculate the average waiting time (avg_wt) and average turnaround time (avg_tat) using the formulas:

$$\text{avg_wt} = \text{wt} * 1.0 / \text{NOP} \quad \text{avg_tat} = \text{tat} * 1.0 / \text{NOP}$$

Step 10: Display the average waiting time and average turnaround time.

Step 11: End

PROGRAM:

```
#include<stdio.h>
int main()
{
    int i, NOP, sum=0, count=0, y, quant, wt=0, tat=0, at[10], bt[10], temp[10];
    float avg_wt, avg_tat;
    printf(" Total number of process in the system: ");
    scanf("%d", &NOP);
    y = NOP; // Assign the number of process to variable y

    for(i=0; i<NOP; i++)
    {
        printf("\n Enter the Arrival and Burst time of the Process[%d]\n", i+1);
        printf(" Arrival time is: \t");
        scanf("%d", &at[i]);
        printf(" \nBurst time is: \t");
        scanf("%d", &bt[i]);
        temp[i] = bt[i];
    }
    printf("Enter the Time Quantum for the process: \t");
    scanf("%d", &quant);
    printf("\n Process No \t\t Burst Time \t\t TAT \t\t Waiting Time ");
    for(sum=0, i = 0; y!=0; )
    {
        if(temp[i] <= quant && temp[i] > 0)
        {
            sum = sum + temp[i];
            temp[i] = 0;
            count=1;
        }
        else if(temp[i] > 0)
        {
            temp[i] = temp[i] - quant;
            sum = sum + quant;
        }
        if(temp[i]==0 && count==1)
        {
            y--;
        }
    }
}
```

```

        printf("\nProcess No[%d] \t\t %d\t\t\t\t %d\t\t\t %d", i+1, bt[i], sum-at[i], sum-at[i]-
bt[i]);
        wt = wt+sum-at[i]-bt[i];
        tat = tat+sum-at[i];
        count =0;
    }
    if(i==NOP-1)
    {
        i=0;
    }
    else if(at[i+1]<=sum)
    {
        i++;
    }
    else
    {
        i=0;
    }
}
avg_wt = wt * 1.0/NOP;
avg_tat = tat * 1.0/NOP;
printf("\n Average Waiting Time: \t%f", avg_wt);
printf("\n Average Turnaround Time: \t%f", avg_tat);
return 0;
}

```

OUTPUT:

```
Total number of process in the system: 4

Enter the Arrival and Burst time of the Process[1]
Arrival time is:      0
Burst time is:  8

Enter the Arrival and Burst time of the Process[2]
Arrival time is:      1
Burst time is:  5

Enter the Arrival and Burst time of the Process[3]
Arrival time is:      2
Burst time is: 10

Enter the Arrival and Burst time of the Process[4]
Arrival time is:      3
Burst time is: 11
Enter the Time Quantum for the process:      6
```

Process No	Burst Time	TAT	Waiting Time
Process No[2]	5	10	5
Process No[1]	8	25	17
Process No[3]	10	27	17
Process No[4]	11	31	20

```
Average Turn Around Time:      14.750000
Average Waiting Time:  23.250000
```

RESULT:

Thus the program is executed successfully.

Ex.No:5.1

IMPLEMENTATION OF SHARED MEMORY IPC

DATE :

AIM:

To write a c program to develop an application using Inter process Communication (IPC) using shared memory IPC

ALGORITHM:

SERVER:

1. Define a character variable c.
2. Define an integer variable shmid to store the shared memory ID.
3. Define a key_t variable key and set it to 2013.
4. Define a character pointer variable shm and a char pointer variable s.
5. Use the shmget() function to create a shared memory segment with the key and size specified in shmsize. If shmget() returns a negative value, output an error message using perror() and exit the program using exit(1).
6. Print the shared memory ID using printf().
7. Use the shmat() function to attach the shared memory segment. If shmat() returns a negative value, output an error message using perror() and exit the program using exit(1).
8. Use memset() to set the shared memory to 0.
9. Set s equal to shm.
10. Use a for loop to write the characters 'a' through 'z' onto the shared memory, incrementing s each time.
11. Add a null character at the end of the shared memory.
12. Use a while loop to wait for the client to finish reading the shared memory by checking if the first character of the shared memory is equal to '*'.
13. Print a message indicating that the client has finished reading.
14. Use shmdt() to detach the shared memory. If shmdt() returns a non-zero value, output an error message using fprintf().
15. Use shmctl() to remove the shared memory segment.

CLIENT:

1. Define an integer variable shmid to store the shared memory ID.

2. Define a `key_t` variable `key` and set it to 2013.
3. Define a character pointer variable `shm` and a character pointer variable `s`.
4. Use the `shmget()` function to get the shared memory segment with the key and size specified in `shmsize`. If `shmget()` returns a negative value, output a message indicating that the server is not started using `printf()` and exit the program using `exit(1)`.
5. Print the shared memory ID using `printf()`.
6. Use the `shmat()` function to attach the shared memory segment. If `shmat()` returns a negative value, output an error message using `perror()` and exit the program using `exit(1)`.
7. Use a for loop to read the characters from the shared memory and print them to the screen, incrementing `s` each time.
8. Add a newline character to the end of the output.
9. Set the first character of the shared memory to '*' to indicate that the client has finished reading.
10. Use `shmdt()` to detach the shared memory. If `shmdt()` returns a non-zero value, output an error message using `fprintf()`.

PROGRAM

SERVER:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/un.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define shmsize 27
int main()
{
    char c;
    int shmid;
    key_t key = 2013;
    char *shm, *s;
    if ((shmid = shmget(key, shmsize, IPC_CREAT|0666)) < 0)
    {
        perror("shmget");
        exit(1);
    }
    printf("Shared memory id : %d\n", shmid);
    if ((shm = shmat(shmid, NULL, 0)) == (char *) -1)
    {
        perror("shmat");
        exit(1);
    }
```



```

}
memset(shm, 0, shmsize);
s = shm;
printf("Writing (a-z) onto shared memory\n");
for (c = 'a'; c <= 'z'; c++)
*s++ = c;
*s = '\0';
while (*shm != '*');
printf("Client finished reading\n");
if(shmdt(shm) != 0)
fprintf(stderr, "Could not close memory segment.\n");
shmctl(shmid, IPC_RMID, 0);
return 0;
}

```

CLIENT:

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define shmsize 27
int main()
{
int shmid;
key_t key = 2013;
char *shm, *s;
if ((shmid = shmget(key, shmsize, 0666)) < 0)
{
printf("Server not started\n");
exit(1);
}
else
printf("Accessing shared memory id : %d\n",shmid);
if ((shm = shmat(shmid, NULL, 0)) == (char *) -1)
{
perror("shmat");
exit(1);
}
printf("Shared memory contents:\n");
for (s = shm; *s != '\0'; s++)
putchar(*s);
putchar('\n');
*shm = '*';
return 0;
}

```

OUTPUT:**SERVER:**

```
[2cs254@cc5 ~]$ vi sserver.c
```

```
[2cs254@cc5 ~]$ cc sserver.c
```

```
[2cs254@cc5 ~]$ ./a.out
```

Shared memory id : 458756

Writing (a-z) onto shared memory

Client finished reading

CLIENT:

```
[2cs254@cc5 ~]$ vi client.c
```

```
[2cs254@cc5 ~]$ cc client.c
```

```
[2cs254@cc5 ~]$ ./a.out
```

shared memory id:458756

shared memory contents:

abcdefghijklmnopqrstuvwxyz

RESULT:

Thus the program is executed successfully.

Ex.No:6

**IMPLEMENTATION OF MUTUAL EXCLUSION BY
SEMAPHORE**

DATE :

AIM:

To write a C program to implement the Producer & consumer Problem (Semaphore)

ALGORITHM:

Step 1: The Semaphore mutex, full & empty are initialized.

Step 2: In the case of producer process

- i) Produce an item in to temporary variable.
- ii) If there is empty space in the buffer check the mutex value for enter into the critical section.
- iii) If the mutex value is 0, allow the producer to add value in the temporary variable to the buffer.

Step 3: In the case of consumer process

- i) It should wait if the buffer is empty
- ii) If there is any item in the buffer check for mutex value, if the mutex==0, remove item from buffer
- iii) Signal the mutex value and reduce the empty value by 1.
- iv) Consume the item.

Step 4: Print the result

PROGRAM:

```
#include<stdio.h>
int mutex=1,full=0,empty=3,x=0;
int main()
{
int n;
int producer();
int consumer();
int wait(int);
int signal(int);
printf("\n1.Producer \n2.Consumer \n3.Exit\n");
while(1)
{
printf("\n Enter your choice\n");
scanf("%d",&n);
switch(n)
{
```

```

case 1:
if((mutex==1)&&(empty!=0))
producer();
else
printf("\n Buffer is Full\n");
break;
case 2:
if((mutex==1)&&(full!=0))
consumer();
else
printf("\n Buffer is Empty\n");
break;
case 3:
exit(0);
break;
}
}
}
int wait(int s)
{
return(--s);
}
int signal(int s)
{
return(++s);
}
producer()
{
mutex=wait(mutex);
full=signal(full);
empty=wait(empty);
x++;
printf("\n Producer Produces the item%d\n",x);
mutex=signal(mutex);
}
consumer()
{
mutex=wait(mutex);
full=wait(full);
empty=signal(empty);
printf("\n Consumer consumes item%d\n",x);
x--;
mutex=signal(mutex);
}

```

OUTPUT:

```
[2cs163@cc5]$ vi semaphore.c
```

```
[2cs163@cc5]$ cc semaphore.c
```

```
[2cs163@cc5]$ ./a.out
```

```
1.Producer
```

```
2.Consumer
```

```
3.Exit
```

```
Enter your choice
```

```
1
```

```
Producer Produces the item1
```

```
Enter your choice
```

```
2
```

```
Consumer consumes item1
```

```
Enter your choice
```

```
3
```

```
[2cs163@cc5 ~]$ ./a.out
```

```
1.Producer
```

```
2.Consumer
```

```
3.Exit
```

```
Enter your choice
```

```
2
```

```
Buffer is Empty
```

```
Enter your choice
```

```
3
```

RESULT:

Thus the program is executed successfully.

Ex.No:7.1

BANKERS ALGORITHM FOR DEADLOCK AVOIDANCE

DATE :

AIM:

To implement the bankers algorithm for Dead Lock Avoidance.

ALGORITHM:

- Step 1. Start the program.
- Step 2. Get the values of resources and processes.
- Step 3. Get the avail value.
- Step 4. After allocation find the need value.
- Step 5. Check whether it's possible to allocate.
- Step 6. If it is possible then the system is in safe state.
- Step 7. Else system is not in safety state.
- Step 8. If the new request comes then check that the system is in safety.
- Step 9. or not if we allow the request.
- Step 10. stop the program.
- Step 11. end

PROGRAM:

```
#include<stdio.h>
int max[100][100];
int alloc[100][100];
int need[100][100];
int avail[100];
int n,r;
void input();
void show();
void cal();
int main()
{
    int i,j;
    printf("***** Banker's Algorithm *****\n");
    input();
    show();
    cal();
    return 0;
}
void input()
{
    int i,j;
    printf("Enter the no of Processes\t");
```

```

scanf("%d",&n);
printf("Enter the no of resources instances\t");
scanf("%d",&r);
printf("Enter the Max Matrix\n");
for(i=0;i<n;i++)
{
    for(j=0;j<r;j++)
    {
        scanf("%d",&max[i][j]);
    }
}
printf("Enter the Allocation Matrix\n");
for(i=0;i<n;i++)
{
    for(j=0;j<r;j++)
    {
        scanf("%d",&alloc[i][j]);
    }
}
printf("Enter the available Resources\n");
for(j=0;j<r;j++)
{
    scanf("%d",&avail[j]);
}
}
void show()
{
    int i,j;
    printf("Process\t Allocation\t Max\t Available\t");
    for(i=0;i<n;i++)
    {
        printf("\nP%d\t ",i+1);
        for(j=0;j<r;j++)
        {
            printf("%d ",alloc[i][j]);
        }
        printf("\t");
        for(j=0;j<r;j++)
        {
            printf("%d ",max[i][j]);
        }
        printf("\t");
        if(i==0)
        {
            for(j=0;j<r;j++)
            printf("%d ",avail[j]);
        }
    }
}
void cal()
{
    int finish[100],temp,need[100][100],flag=1,k,c1=0;

```

```

int safe[100];
int i,j;
for(i=0;i<n;i++)
{
    finish[i]=0;
}

for(i=0;i<n;i++)
{
    for(j=0;j<r;j++)
    {
        need[i][j]=max[i][j]-alloc[i][j];
    }
}
printf("\n");
while(flag)
{
    flag=0;
    for(i=0;i<n;i++)
    {
        int c=0;
        for(j=0;j<r;j++)
        {
            if((finish[i]==0)&&(need[i][j]<=avail[j]))
            {
                c++;
                if(c==r)
                {
                    for(k=0;k<r;k++)
                    {
                        avail[k]+=alloc[i][j];
                        finish[i]=1;
                        flag=1;
                    }
                    printf("P%d->",i);
                    if(finish[i]==1)
                    {
                        i=n;
                    }
                }
            }
        }
    }
}
for(i=0;i<n;i++)
{
    if(finish[i]==1)
    {
        c1++;
    }
    else
    {

```



```
    printf("P%d->",i);
    }
}
if(c1==n)
{
    printf("\n The system is in safe state");
}
else
{
    printf("\n Process are in dead lock");
    printf("\n System is in unsafe state");
}
}
```

OUT PUT:

[2cs121@cc5 ~]\$ vi dead.c

[2cs121@cc5 ~]\$ cc dead.c

[2cs121@cc5 ~]\$./a.out

***** Banker's Algorithm *****

Enter the no of Processes 5

Enter the no of resources instances 3

Enter the Max Matrix

7 5 6

3 2 2

9 0 2

2 2 2

4 3 3

Enter the Allocation Matrix

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter the available Resources

3 3 2

Process	Allocation	Max	Available
---------	------------	-----	-----------

P1	0 1 0	7 5 6	3 3 2
----	-------	-------	-------

P2	2 0 0	3 2 2	
----	-------	-------	--

P3	3 0 2	9 0 2	
----	-------	-------	--

P4	2 1 1	2 2 2	
----	-------	-------	--

P5	0 0 2	4 3 3	
----	-------	-------	--

P1->P3->P4->P2->P0->

The system is in safe state[2cs121@cc5 ~]\$

RESULT:

Thus the program is executed successfully.

Ex.No:7.2

IMPLEMENTATION OF DEADLOCK DETECTION ALGORITHM

DATE :

AIM:

To write a C Program to implement deadlock Detection technique.

ALGORITHM:

Step 1. Start the program.

Step 2. Get the values of resources and processes.

Step 3. Get the avail value.

Step 4. After allocation find the need value.

Step 5. Check whether it's possible to allocate.

Step 6. If it is possible then the system is in safe state.

Step 7. Else system is not in safety state

Step 8. Stop the process.

PROGRAM:

```
#include<stdio.h>
int max[100][100];
int alloc[100][100];
int need[100][100];
int avail[100];
int n,r;
void input();
void show();
void cal();
int main()
{
    int i,j;
    printf("***** Deadlock Detection Algo *****\n");
    input();
    show();
    cal();
    return 0;
}
void input()
{
    int i,j;
    printf("Enter the no of Processes\t");
```

```

scanf("%d",&n);
printf("Enter the no of resource instances\t");
scanf("%d",&r);
printf("Enter the Max Matrix\n");
for(i=0;i<n;i++)
{
    for(j=0;j<r;j++)
    {
        scanf("%d",&max[i][j]);
    }
}
printf("Enter the Allocation Matrix\n");
for(i=0;i<n;i++)
{
    for(j=0;j<r;j++)
    {
        scanf("%d",&alloc[i][j]);
    }
}
printf("Enter the available Resources\n");
for(j=0;j<r;j++)
{
    scanf("%d",&avail[j]);
}
}
void show()
{
    int i,j;
    printf("Process\t Allocation\t Max\t Available\t");
    for(i=0;i<n;i++)
    {
        printf("\nP%d\t ",i+1);
        for(j=0;j<r;j++)
        {
            printf("%d ",alloc[i][j]);
        }
        printf("\t");
        for(j=0;j<r;j++)
        {
            printf("%d ",max[i][j]);
        }
        printf("\t");
        if(i==0)
        {
            for(j=0;j<r;j++)
            printf("%d ",avail[j]);
        }
    }
}
void cal()
{
    int finish[100],temp,need[100][100],flag=1,k,c1=0;

```

```

int dead[100];
int safe[100];
int i,j;
for(i=0;i<n;i++)
{
    finish[i]=0;
}
for(i=0;i<n;i++)
{
    for(j=0;j<r;j++)
    {
        need[i][j]=max[i][j]-alloc[i][j];
    }
}
while(flag)
{
    flag=0;
    for(i=0;i<n;i++)
    {
        int c=0;
        for(j=0;j<r;j++)
        {
            if((finish[i]==0)&&(need[i][j]<=avail[j]))
            {
                c++;
                if(c==r)
                {
                    for(k=0;k<r;k++)
                    {
                        avail[k]+=alloc[i][j];
                        finish[i]=1;
                        flag=1;
                    }
                    if(finish[i]==1)
                    {
                        i=n;
                    }
                }
            }
        }
    }
}
j=0;
flag=0;
for(i=0;i<n;i++)
{
    if(finish[i]==0)
    {
        dead[j]=i;
        j++;
        flag=1;
    }
}

```

```
    }
    if(flag==1)
    {
        printf("\n\nSystem is in Deadlock and the Deadlock process are\n");
        for(i=0;i<n;i++)
        {
            printf("P%d\t",dead[i]);
        }
    }
    else
    {
        printf("\nNo Deadlock Occur");
    }
}
```

OUTPUT:

```
[2cs254@cc5 ~]$ vi deadd.c
[2cs254@cc5 ~]$ cc deadd.c
[2cs254@cc5 ~]$ ./a.out
***** Deadlock Detection Algo *****
Enter the no of Processes      3
Enter the no of resource instances  3
Enter the Max Matrix
3 6 8
4 3 3
3 4 4
Enter the Allocation Matrix
3 3 3
2 0 3
1 2 4
Enter the available Resources
1 2 0
Process Allocation    Max    Available
P1    3 3 3    3 6 8  1 2 0
P2    2 0 3    4 3 3
P3    1 2 4    3 4 4

System is in Deadlock and the Deadlock process are
P0    P1    P2
[2cs254@cc5 ~]$:
```

RESULT:

Thus the program is executed successfully.

EX. NO. 8

DATE:

THREADING & SYNCHRONIZATION APPLICATIONS

AIM:

To implement a C program for threading and synchronization application.

ALGORITHM:

Step 1: Start the program.

Step 2: Create two threads.

Step 3: A counter is maintained through which user gets the logs about job number which is started and when it is completed

Step 4: If we focus on the last two logs, we will see that the log 'Job 2 finished' is repeated twice while no log for 'Job 1 finished' is seen.

Step 5: if we go back at the code and try to find any logical flaw, you'll probably not find any flaw easily

Step 6: if we will have a closer look and visualize the execution of the code, you'll find following

- The log 'Job 2 started' is printed just after 'Job 1 Started' so it can easily be concluded that while thread 1 was processing the scheduler scheduled the thread 2.
- If the above assumption was true then the value of the 'counter' variable got incremented again before job 1 got finished.
- So, when Job 1 actually got finished, then the wrong value of counter produced the log 'Job 2 finished' followed by the 'Job 2 finished' for the actual job 2 or vice versa as it is dependent on scheduler.
- So we see that its not the repetitive log but the wrong value of the 'counter' variable that is the problem.

Step 7: Exit the program.

PROGRAM:

```
#include<stdio.h>
#include<string.h>
#include<pthread.h>
#include<stdlib.h>
#include<unistd.h>
pthread_t tid[2];
int counter;
void* doSomething(void *arg)
{
    unsigned long i = 0;
    counter += 1;
    printf("\n Job %d started\n", counter);
```

```
    for(i=0; i<(0xFFFFFFFF);i++);
    printf("\n Job %d finished\n", counter);
    return NULL;
}
int main(void)
{
    int i = 0;
    int err;
    while(i < 2)
    {
        err = pthread_create(&(tid[i]), NULL, &doSomething, NULL);
        if (err != 0)
            printf("\ncan't create thread :[%s]", strerror(err));
        i++;
    }
    pthread_join(tid[0], NULL);
    pthread_join(tid[1], NULL);
    return 0;
}
```

OUTPUT:

```
[it@localhost ~]$ ~]$ ./a.out
```

Job 1 started

Job 2 started

Job 2 finished

Job 2 finished

RESULT:

Thus the program is executed successfully.

EX. NO. 9.1

DATE:

IMPLEMENTATION OF FIRST FIT MEMORY ALLOCATION METHODS FOR FIXED PARTITION

AIM:

To write a program to implement first fit Memory Allocation Methods for fixed partition.

ALGORITHM:

1. Get no. of files and no. of blocks.
2. After that get the size of each block and file requests.
3. Now allocate file
if(block size $\geq$ file size) then allocate the file otherwise move on to next block
4. Display the file with the blocks that are allocated to a respective files.
5. Stop.

PROGRAM:

```
#include<stdio.h>
#define max 25
void main()
{
    int frag[max],b[max],f[max],i,j,nb,nf,temp;
    static int bf[max],ff[max];
    printf("\n\tMemory Management Scheme - First Fit");
    printf("\nEnter the number of blocks:");
    scanf("%d",&nb);
    printf("Enter the number of files:");
    scanf("%d",&nf);
    printf("\nEnter the size of the blocks:-\n");
    for(i=1;i<=nb;i++)
    {
        printf("Block %d:",i);
        scanf("%d",&b[i]);
    }
    printf("Enter the size of the files :-\n");
    for(i=1;i<=nf;i++)
    {
        printf("File %d:",i);
        scanf("%d",&f[i]);
    }
    for(i=1;i<=nf;i++)
    {
        for(j=1;j<=nb;j++)
        {
```

```
        if(bf[j]!=1)
        {
            temp=b[j]-f[i];
            if(temp>=0)
            {
                ff[i]=j;
                break;
            }
        }
        frag[i]=temp;
        bf[ff[i]]=1;
    }
    printf("\nFile_no:\tFile_size :\tBlock_no:\tBlock_size:\tFragement");
    for(i=1;i<=nf;i++)
        printf("\n%d\t%d\t%d\t%d\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
}
```

OUTPUT:

INPUT

Enter the number of blocks: 3

Enter the number of files: 2

Enter the size of the blocks:-

Block 1: 5

Block 2: 2

Block 3: 7

Enter the size of the files:-

File 1: 1

File 2: 4

OUTPUT

File No	File Size	Block No	Block Size	Fragment
1	1	1	5	4
2	4	3	7	3

RESULT:

Thus the program is executed successfully.

EX. NO. 9.2**DATE:****WORST FIT MEMORY ALLOCATION METHODS FOR FIXED
PARTITION****AIM:**

To write a program to implement worst fit Memory Allocation Methods for fixed partition.

ALGORIHTM:

1. Get no. of files and no. of blocks.
2. After that get the size of each block and file requests.
3. Now allocate file to the large size block
4. Display the file with the blocks that are allocated to a respective file.
5. Stop.

PROGRAM:

```
#include<stdio.h>
#define max 25
void main()
{
    int frag[max],b[max],f[max],i,j,nb,nf,temp,highest=0;
    static int bf[max],ff[max];
    printf("\n\tMemory Management Scheme - Worst Fit");
    printf("\n\tEnter the number of blocks:");
    scanf("%d",&nb);
    printf("Enter the number of files:");
    scanf("%d",&nf);
    printf("\n\tEnter the size of the blocks:-\n");
    for(i=1;i<=nb;i++)
    {
        printf("Block %d:",i);
        scanf("%d",&b[i]);
    }
    printf("Enter the size of the files :-\n");
    for(i=1;i<=nf;i++)
    {
        printf("File %d:",i);
        scanf("%d",&f[i]);
    }
    for(i=1;i<=nf;i++)
    {
        for(j=1;j<=nb;j++)
```

```

        {
            if(bf[j]!=1) //if bf[j] is not allocated
            {
                temp=b[j]-f[i];
                if(temp>=0)
                    if(highest<temp)
                    {
                        ff[i]=j;
                        highest=temp;
                    }
            }
        }
        frag[i]=highest;
        bf[ff[i]]=1;
        highest=0;
    }
    printf("\nFile_no:\tFile_size :\tBlock_no:\tBlock_size:\tFragement");
    for(i=1;i<=nf;i++)
        printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
}

```


OUTPUT:

Memory Management Scheme - Worst Fit

Enter the number of blocks: 3

Enter the number of files: 2

Enter the size of the blocks:-

Block 1: 5

Block 2: 2

Block 3: 7

Enter the size of the files:-

File 1: 1

File 2: 4

OUTPUT

File No	File Size	Block No	Block Size	Fragment
1	1	3	7	6
2	4	1	5	1

RESULT

Thus the program is executed successfully.

EX. NO. 9.3

DATE:

BEST FIT MEMORY ALLOCATION METHODS FOR FIXED PARTITION

AIM:

To write a program to implement best fit Memory Allocation Methods for fixed partition.

ALGORITHM:

1. Get no. of files and no. of blocks.
2. After that get the size of each block and file requests.
3. Now allocate file to the best size block
4. Display the file with the blocks that are allocated to a respective file.
5. Stop.

PROGRAM:

```
#include<stdio.h>
#define max 25
void main()
{
    int frag[max],b[max],f[max],i,j,nb,nf,temp,lowest=10000;
    static int bf[max],ff[max];
    printf("\nEnter the number of blocks:");
    scanf("%d",&nb);
    printf("Enter the number of files:");
    scanf("%d",&nf);
    printf("\nEnter the size of the blocks:-\n");
    for(i=1;i<=nb;i++)
    {
        printf("Block %d:",i);
        scanf("%d",&b[i]);
    }
    printf("Enter the size of the files :-\n");
    for(i=1;i<=nf;i++)
    {
        printf("File %d:",i);
        scanf("%d",&f[i]);
    }
    for(i=1;i<=nf;i++)
    {
        for(j=1;j<=nb;j++)
        {
            if(bf[j]!=1)
            {
                temp=b[j]-f[i];
                if(temp>=0)
```

```

        if(lowest>temp)
        {
            ff[i]=j;

            lowest=temp;
        }
    }
    frag[i]=lowest;
    bf[ff[i]]=1;
    lowest=10000;
}
printf("\nFile No\tFile Size \tBlock No\tBlock Size\tFragment");
for(i=1;i<=nf && ff[i]!=0;i++)
    printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
}

```

OUTPUT:

Enter the number of blocks: 3

Enter the number of files: 2

Enter the size of the blocks:-

Block 1: 5

Block 2: 2

Block 3: 7

Enter the size of the files:-

File 1: 1

File 2: 4

File No	File Size	Block No	Block Size	Fragment
1	1	2	2	1
2	4	1	5	1

RESULT:

Thus the program is executed successfully.

EX. NO. 10**DATE:****PAGING TECHNIQUE****AIM:**

To write a program to implement paging concept for memory management.

ALGORIHTM:

Step 1: Start the program.

Step 2: Enter the logical memory address.

Step 3: Enter the page table which has offset and page frame.

Step 4: The corresponding physical address can be calculate by,

$PA = [\text{pageframe} * \text{No. of page size}] + \text{Page offset}.$

Step 5: Print the physical address for the corresponding logical address.

Step 6: Terminate the program.

PROGRAM:

```
#include<stdio.h>
#define MAX 50
main()
{
    int page[MAX],i,n,f,ps,off,pno;
    printf("\nEnter the no of pages in memory");
    scanf("%d",&n);
    printf("\nEnter page size");
    scanf("%d",&ps);
    printf("\nEnter no of frames");
    scanf("%d",&f);
    for(i=0;i<n;i++)
        page[i]=-1;
    printf("\nEnter the page table\n");
    printf("(Enter frame no as -1 if that page is not present in any frame)\n\n");
    printf("\npageno\tframeno\n-----\t----- ");
    for(i=0;i<n;i++)
    {
        printf("\n\n%d\t\t",i);
        scanf("%d",&page[i]);
    }
    printf("\n\nEnter the logical address(i.e,page no & offset:");
    scanf("%d%d",&pno,&off);
```

```
if(page[pno]==-1)
    printf("\n\nThe required page is not available in any of frames");
else
    printf("\n\nPhysical address(i.e,frame no & offset):%d,%d",page[pno],off);
}
```

OUTPUT:

```
[it@localcost ~]$ cc paging.c
```

```
[it@localcost ~]$ ./a.out
```

Enter the no.of pages in memory : 3

Enter page size : 4

Enter no of frames: 6

Enter the page table

Enter frame no as -1 if that page is not present in any frame

page no	frame no
0	3
1	2
2	5

Enter the logical address(i.e,page no & offset): 1 3

Physical address(i.e,frame no & offset):2 3

RESULT

Thus the program is executed successfully.

Ex. No: 11.1**FIFO PAGE REPLACEMENT ALGORITHM****AIM:**

To implement page replacement algorithms FIFO (First In First Out)

ALGORITHM:

Step1: Start the Program

Step 2: Create a queue to hold all pages in memory

Step 3: When the page is required replace the page at the head of the queue

Step 4: Now the new page is inserted at the tail of the queue

Step5: display the result

Step6 : Stop the program

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
int i,j,nof,nor,flag=0,ref[50],frm[50],pf=0,victim=-1;
void main()
{
    printf("\n \t\t\t FIFI PAGE REPLACEMENT ALGORITHM");
    printf("\n Enter no.of frames... ");
    scanf("%d",&nof);
    printf("Enter number of reference string..\n");
    scanf("%d",&nor);
    printf("\n Enter the reference string..");
    for(i=0;i<nor;i++)
        scanf("%d",&ref[i]);
    printf("\nThe given reference string:");
    for(i=0;i<nor;i++)
        printf("%4d",ref[i]);
    for(i=1;i<=nof;i++)
        frm[i]=-1;
    printf("\n");
    for(i=0;i<nor;i++)
    {
        flag=0;
        printf("\n\t Reference np%d->\t",ref[i]);
        for(j=0;j<nof;j++)
        {
            if(frm[j]==ref[i])
            {
                flag=1;
            }
        }
    }
}
```



```
                break;
            }
        }
    if(flag==0)
    {
        pf++;
        victim++;
        victim=victim%nof;
        frm[victim]=ref[i];
        for(j=0;j<nof;j++)
            printf("%4d",frm[j]);
    }
}
printf("\n\n\t\t No.of pages faults...%d",pf);
}
```

OUTPUT:

Enter no.of frames... 4

Enter number of reference string..

6

Enter the reference string..

5 6 4 1 2 3

The given reference string:

..... 5 6 4 1 2 3

Reference np5-> 5 -1 -1 -1

Reference np6-> 5 6 -1 -1

Reference np4-> 5 6 4 -1

Reference np1-> 5 6 4 1

Reference np2-> 2 6 4 1

Reference np3-> 2 3 4 1

No.of pages faults...6

RESULT:

Thus the program is executed successfully.

Ex. No:11.2

LRU PAGE REPLACEMENT ALGORITHM

AIM:

To implement page replacement algorithm LRU (Least Recently Used)

ALGORITHM:

Step1: Start the Program

Step 2: Create a queue to hold all pages in memory

Step 3: When the page is required replace the page at the head of the queue

Step 4: Now the new page is inserted at the tail of the queue

Step 5: Create a stack

Step 6: When the page fault occurs replace page present at the bottom of the stack

Step 7: display the result

Step8 : Stop the program

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
int i,j,nof,nor,flag=0,ref[50],frm[50],pf=0,victim=-1;
int recent[10],lrucal[50],count=0;
int lruvictim();

void main()
{
    printf("\n\t\t\t LRU PAGE REPLACEMENT ALGORITHM");
    printf("\n Enter no.of Frames... ");
    scanf("%d",&nof);
    printf(" Enter no.of reference string..");
    scanf("%d",&nor);
    printf("\n Enter reference string..");
    for(i=0;i<nor;i++)
        scanf("%d",&ref[i]);
    printf("\n\n\t\t LRU PAGE REPLACEMENT ALGORITHM ");
    printf("\n\t The given reference string:");
    printf("\n..... ");
    for(i=0;i<nor;i++)
        printf("%4d",ref[i]);
    for(i=1;i<=nof;i++)
    {
```

```

        frm[i]=-1;
        lrucal[i]=0;
    }
    for(i=0;i<10;i++)
        recent[i]=0;
    printf("\n");
    for(i=0;i<nor;i++)
    {
        flag=0;
        printf("\n\t Reference NO %d->\t",ref[i]);
        for(j=0;j<nof;j++)
        {
            if(frm[j]==ref[i])
            {
                flag=1;
                break;
            }
        }
        if(flag==0)
        {
            count++;
            if(count<=nof)
                victim++;
            else
                victim=lruvictim();
            pf++;
            frm[victim]=ref[i];
            for(j=0;j<nof;j++)
                printf("%4d",frm[j]);
        }
        recent[ref[i]]=i;
    }
    printf("\n\n\t No.of page faults...%d",pf);
    getch();
}

int lruvictim()
{
    int i,j,temp1,temp2;
    for(i=0;i<nof;i++)
    {
        temp1=frm[i];
        lrucal[i]=recent[temp1];
    }
    temp2=lrucal[0];
    for(j=1;j<nof;j++)
    {
        if(temp2>lrucal[j])

```

```
        temp2=lrucal[j];
    }
    for(i=0;i<nof;i++)
        if(ref[temp2]==frm[i])
            return i;
    return 0;
}
```

OUTPUT:

Enter no.of Frames... 3

Enter no.of reference string 6

Enter reference string..

6 5 4 2 3 1

LRU PAGE REPLACEMENT ALGORITHM

The given reference string:

..... 6 5 4 2 3 1

Reference NO 6-> 6 -1 -1

Reference NO 5-> 6 5 -1

Reference NO 4-> 6 5 4

Reference NO 2-> 2 5 4

Reference NO 3-> 2 3 4

Reference NO 1-> 2 3 1

No.of page faults...6

RESULT:

Thus the program is executed successfully.

EX. NO. 12 .1**DATE:****FILE ORGANIZATION TECHNIQUES - SINGLE LEVEL DIRECTORY****AIM:**

To implement a program for single level directory file organization techniques.

ALGORITHM:

1. Start the program.
2. Create a single directory.
3. Enter the no of files and file names for the particular directory.
4. Now the Directory consist of all files .
5. Display the result using graphics.
6. Stop the program

PROGRAM:

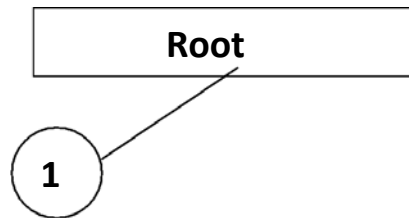
```
#include<stdio.h>
#include<stdlib.h>
#include<graphics.h>
void main()
{
    int gd = DETECT, gm, count, i,j, mid,cir_x;
    char fname[20][30];
    initgraph(&gd, &gm,"c:\\tc\\bgi");
    cleardevice();
    setbkcolor(GREEN);
    puts("Enter no of files u have ");
    scanf("%d", &count);
    for(i=0;i<count;i++)
    {
        cleardevice();
        setbkcolor(GREEN);
        printf("Enter File %d name", i+1);
        scanf("%s",fname[i]);
        setfillstyle(1,MAGENTA);
        mid=640/count;
        cir_x=mid/3;
        bar3d(270,100,370,150,0,0);
        settextstyle(2,0,4);
        settextjustify(1,1);
        outtextxy(320,125,"Root Directory");
        setcolor(BLUE);
        for(j=0;j<=i;j++,cir_x+=mid)
```

```
        {
            line(320,150,cir_x,250);
            fillellipse(cir_x,250,30,30);
            outtextxy(cir_x,250,fname[j]);
        }
        getch();
    }
}
```

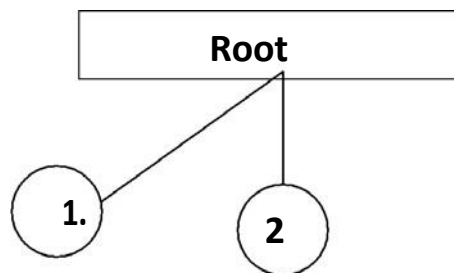

OUTPUT:

Enter no of filesdouhave?3

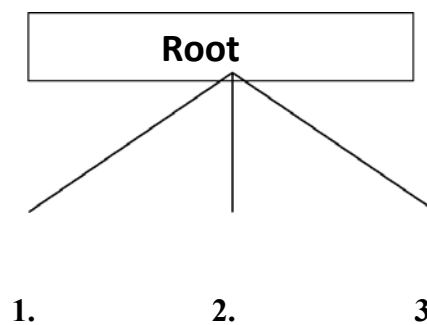
Enterfile1name:1.c



EnterFile2name:2.c



EnterFile3name:3.c



RESULT:

Thus the program is executed successfully.

Ex. No. 12.2

DATE:

FILE ORGANIZATION TECHNIQUES - TWO LEVEL DIRECTORY

AIM:

To implement a program for two level directory file organization techniques.

ALGORITHM:

1. Start the program.
2. Each user create a private directory
3. Enter the no of user and file name for particular directory, In this way names chosen by one user don't interface with names chosen by a different user.
4. Display the result using graphics.
5. Stop the program

PROGRAM:

```
#include<stdio.h>
#include<graphics.h>
struct tree_element
{
char name[20];
intx,y,ftype,lx,rx,nc,level;
struct tree_element *link[5];
};
typedef struct tree_element node;
void main()
{
intgd=DETECT,gm;
node*root;
root=NULL;
clrscr();
create(&root,0,"null",0,639,320);clrscr();
initgraph(&gd,&gm,"c:\\tc\\bgi");display(root);
getch();
closegraph();
}
create(node** root,int lev,char*dname,int lx,int rx,int x)
{
int i,gap;
if(*root== NULL)
{
(*root)=(node*)malloc(sizeof(node));
printf("enternameofdir/file(under %s):",dname);
```

```

fflush(stdin);
gets((*root)->name);
if(lev==0||lev==1)
(*root)->ftype=1;
else
(*root)->ftype=2;
(*root)->level=lev;

(*root)->y=50+lev*50;
(*root)->x=x;
(*root)->lx=lx;
(*root)->rx=rx;
for(i=0;i<5;i++)
(*root)->link[i]=NULL;
if((*root)->ftype==1)
{
if(lev==0||lev==1)
{
if((*root)->level==0)
printf("Howmanyusers");else
printf("how many files");
printf("(for%s):",(*root)->name);
scanf("%d",&(*root)->nc);
}
else(*root)->nc=0;
if((*root)->nc==0)
gap=rx-lx;
else
gap=(rx-lx)/(*root)->nc;
for(i=0;i<(*root)->nc;i++)
create(&((*root)->link[i]),lev+1,(*root)->name,lx+gap*i,lx+gap*i+gap,lx+gap*i+gap/2);
}
else
(*root)->nc=0;
}
}
display(node*root)
{
int i;
settextstyle(2,0,4);
settextjustify(1,1);
setfillstyle(1,BLUE);
setcolor(14);
if(root!=NULL)
{
for(i=0;i<root->nc;i++)
{

```

```
line(root->x,root->y,root->link[i]->x,root->link[i]->y);
}
if(root->ftype==1)
bar3d(root->x-20,root->y-10,root->x+20,root->y+10,0,0);
else
    fillellipse(root->x,root->y,20,20);
    outtextxy(root->x,root->y,root->name);
    for(i=0;i<root->nc;i++)
    {
display(root->link[i]);
}
}
}
```

OUTPUT:

enternameofdir/file(undernull):sld

Howmanyusers(forsld):2

enternameofdir/file(undersld):tld

howmanyfiles(fortld):2

enternameofdir/file(undertld):hir

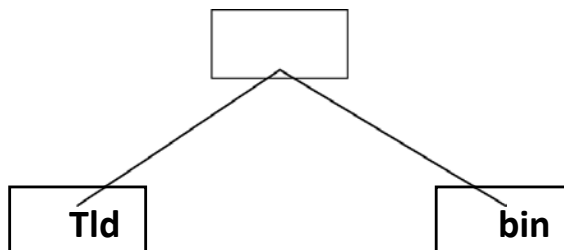
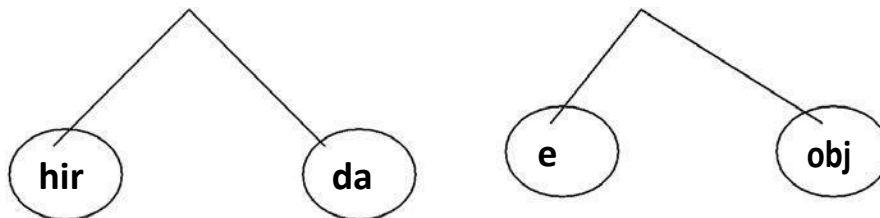
enternameofdir/file(undertld):deg

enternameofdir/file(undersld):bin

hoemanyfiles(forbin):2

enternameofdir/file(underbin):exe

enternameofdir/file(underbin):obj

**RESULT:**

Thus the program is executed successfully.

EX.NO :13.1

DATE:

SEQUENTIAL FILE ALLOCATION STRATEGY

AIM:

To implement a program for Sequential File Allocation Strategy.

ALGORITHM:

1. Start the program.
2. Get the block and length of the file.
3. Check the files are allocated or not.
4. If the files are not allocated, then allocate in a sequential manner.
5. Display the result
6. Stop the program

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
main()
{
    int f[50],i,st,j,len,c,k;
    clrscr();
    for(i=0;i<50;i++)
        f[i]=0;
    X:
    printf("\n Enter the starting block & length of file");
    scanf("%d%d",&st,&len);
    for(j=st;j<(st+len);j++)
        if(f[j]==0)
        {
            f[j]=1;
            printf("\n%d->%d",j,f[j]);
        }
    else
    {
        printf("Block already allocated");
        break;
    }
    if(j==(st+len))
        printf("\n the file is allocated to disk");
    printf("\n if u want to enter more files?(y-1/n-0)");
    scanf("%d",&c);
    if(c==1)
        goto X;
    else
```

```
        exit();  
        getch();  
    }
```

OUTPUT:

Enter the starting block & length of file 4 10

4 -> 1

5 -> 1

6 -> 1

7 -> 1

8 -> 1

9 -> 1

10 -> 1

11 -> 1

12 -> 1

13 -> 1

The file is allocated to disk

if u want to enter more files ? (y-1/n-0)

RESULT:

Thus the program is executed successfully.

EX.NO: 13.2

DATE:

INDEXED FILE ALLOCATION STRATEGY

AIM:

To implement a program for Indexed File Allocation Strategy.

ALGORITHM:

1. Start the program.
2. Enter the index of the blocks & no of files on index.
3. Check the files are allocated or not
4. If the files are not allocated, then allocate the file based on the indexes that are created for each fragment of the file.
5. Display the result
6. Stop the program

PROGRAM:

```
#include<stdio.h>
int f[50],i,k,j,inde[50],n,c,count=0,p;
main()
{
    for(i=0;i<50;i++)
        f[i]=0;
    x:
    printf("enter index block\t");
    scanf("%d",&p);
    if(f[p]==0)
    {
        f[p]=1;
        printf("enter no of files on index\t");
        scanf("%d",&n);
    }
    else
    {
        printf("Block already allocated\n");
        goto x;
    }
    for(i=0;i<n;i++)
        scanf("%d",&inde[i]);
    for(i=0;i<n;i++)
        if(f[inde[i]]==1)
        {
            printf("Block already allocated");
            goto x;
        }
    for(j=0;j<n;j++)
        f[inde[j]]=1;
    printf("\n allocated");
    printf("\n file indexed");
}
```



```
    for(k=0;k<n;k++)
        printf("\n %d->%d:%d",p,inde[k],f[inde[k]]);
    printf(" Enter 1 to enter more files and 0 to exit\t");
    scanf("%d",&c);
    if(c==1)
        goto x;
    else
        exit();

}
```

OUTPUT:

Enter index block 9

Enter no of files on index 3

1 2 3

Allocated

File indexed

9- > 1:1

9- > 2:1

9- > 3:1

Enter 1 to enter more files and 0 to exit

RESULT:

Thus the program is executed successfully.

EX.NO 13.3

DATE:

LINKED FILE ALLOCATION STRATEGY

AIM:

To implement a program for Linked File Allocation Strategy.

ALGORITHM:

1. Start the program.
2. Enter how many blocks & block no that are already allocated.
3. Enter the starting index block & length.
4. If the files are not allocated, then allocate the file based on linked list format where each and every fragment is linked to the other file through either addresses or pointers
5. Display the result
6. Stop the program

PROGRAM:

```
#include<stdio.h>
main()
{
    int f[50],p,i,j,k,a,st,len,n,c;
    for(i=0;i<50;i++)
        f[i]=0;
    printf("Enter how many blocks that are already allocated");
    scanf("%d",&p);
    printf("\nEnter the blocks no.s that are already allocated");
    for(i=0;i<p;i++)
    {
        scanf("%d",&a);
        f[a]=1;
    }
    X:
    printf("Enter the starting index block & length");
    scanf("%d%d",&st,&len);
    k=len;
    for(j=st;j<(k+st);j++)
    {
        if(f[j]==0)
        {
            f[j]=1;
            printf("\n%d->%d",j,f[j]);
        }
        else
        {
            continue;
        }
    }
}
```

```
        printf("\n %d->file is already allocated",j);
        k++;
    }
}
printf("\n If u want to enter one more file? (yes-1/no-0)");
scanf("%d",&c);
if(c==1)
    goto X;
else
    exit();
}
```

OUTPUT:

Enter how many blocks that are already allocated 3

Enter the blocks no.s that are already allocated 4 7 9

Enter the starting index block & length 3 7

3- >1

4- > file is already allocated

5- >1

6- >1

7- > file is already allocated

8- >1

9- > file is already allocated

10 - > 1

11 - > 1

12 - > 1

If u want to enter one more file? (yes-1/no-0)");

RESULT

Thus the program is executed successfully.

EX.NO:14

DATE:

INSTALLING GUEST OPERATING SYSTEM LIKE LINUX USING VMWARE

AIM:

To install any guest operating system like Linux using VMware.

PROCEDURE:

Step-By-Step Procedure To Install Ubuntu Linux On VMWare Workstation

Ubuntu:

[Ubuntu](#) is one of the popular Linux distributions developed on the [Debian Linux](#) platform. Ubuntu is a full operating system available in multiple editions: Desktop edition, Servers edition, IoT, and Cloud. These editions can run on either a computer or virtual machines.

VMWare Workstation:

VMWare Workstation is an application developed by VMWare to create virtual machines, containers, and Kubernetes clusters on any desktop or server system. VMWare released VMWare Workstation in two products: VMWare Workstation Pro and VMWare Workstation Player. VMWare Workstation Player is released on a free license with limited features, whereas VMWare Workstation Pro is an enterprise paid version that has loaded with a lot of features.

Prerequisites To Install Ubuntu On VMWare Workstation:

To install Ubuntu Linux on VMWare Workstation you just need a physical machine, VMWare Workstation Pro or Player, and Ubuntu Linux ISO image.

1. A host system with minimum:
 1. 8 GB of memory
 2. A quad core CPU
 3. 500 GB of Hard Drive
2. VMWare Workstation Pro or Player application:0
 1. Download VMWare Workstation
3. Ubuntu Operating System to install on VMWare Workstation.
 1. Download Ubuntu Linux

How To Install Ubuntu Linux On VMWare Workstation?

Time needed: 30 minutes.

How to Install Ubuntu Linux on VMWare Workstation?

1. Fier up VMWare Workstation

Download the VMWare Workstation application for your host operating system and install it on your machine. The installation procedure is pretty simple and straight. Read the documentation for more details. Open the app after installation. Create a new Virtual Machine.

2. Select Custom Configuration Wizard

You can choose either Typical or Custom Wizard. We recommend selecting Custom if you want to install with all the configurations. If you are okay with default configurations then go ahead with Typical configurations.

3. Select Virtual Machine Hardware Compatibility

Go with the default option if you don't have the choice.

4. Select the Operating System Media

Select 'I will install the operating system later' for an interactive installation.

5. Select Guest Operating System

6. Name the Virtual Machine Name and location

Type a name and give the location details.

7. Allocate the Processors

Assign the processors, Calculate the processor required to run the host machine. Assign the leftover resources to the virtual machine.

8. Allocate the Memory for Virtual Machine

Memory allocation calculation is the same as the processor allocation. Leave sufficient memory for the host system and allocate the remaining memory for the virtual machine.

9. Choose the Network Configuration

Select any one of the network configurations as per your requirement.

10. Select the I/O Controller Type

11. Select Disk Type

12. Select Virtual Disk

Select the Virtual Disk if you have or create one.

13. Select Disk Capacity

Select the disk size. Selecting a single disk will increase the performance. However, selecting a split disk will help in the disk transfer scenario.

14. Specify Virtual Disk File

15. Create Virtual Machine

16. Supply Ubuntu ISO Image to Virtual Machine

Download Ubuntu image. Edit the CD/DVD settings and import the downloaded Ubuntu image.

17. Install Ubuntu Linux on VMWare Workstation

18. Power On the Virtual Machine

Press the Play button to power on the Virtual Machine.

19. Welcome Ubuntu Virtual Machine

After powering on the Virtual machine, you will be treated with a welcome screen on which you will see two options: Try Ubuntu and Install Ubuntu. Select Try Ubuntu if you want to run Ubuntu in live mode. Select Install to continue the installation process.

20. Software Update and Package Selection in Virtual Machine

21. Partition the Disk

Select Erase the Disk for auto partition. Or Select the Advance option to create the custom partition.

22. Write Changes to Disk

23. Select Time Zone

24. Create an Admin Account

25. Installation of Ubuntu in Progress

26. Reboot Virtual Machine

Reboot the machine after installation.

27. Remove the installation media

Remove the installation media before reboot.

28. Boot Ubuntu

RESULT:

Thus the Linux guest operating system using VMware is installed successfully.

CONTENT BEYOND SYLLABUS

MEMORY MANAGEMENT

A). MEMORY MANAGEMENT WITH FIXED PARTITIONING TECHNIQUE (MFT)

AIM: To implement and simulate the MFT algorithm.

ALGORITHM: Step1: Start the process.

Step2: Declare variables. Step3: Enter total memory size ms.

Step4: Allocate memory for os. $M_s = ms - os$

Step5: Read the no partition to be divided n Partition size = ms/n .

Step6: Read the process no and process size.

Step 7: If process size is less than partition size allot else block the process. While allocating update memory wastage-external fragmentation.

```
if(pn[i]==pn[j])f=1;
if(f==0){ if(ps[i]<=siz)
{
extft=extft+sizeps[i];
avail[i]=1;
count++;
}
}
```

Step 8: Print the result

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>
main()
{
    int ms, bs, nob, ef,n,
    mp[10],tif=0; int i,p=0;
    clrscr();
    printf("Enter the total memory available (in Bytes) -- ");
    scanf("%d",&ms);
    printf("Enter the block size (in Bytes) -- ");
    scanf("%d", &bs);
    nob=ms/bs;
    ef=ms - nob*bs;
    printf("\nEnter the number of processes -- ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("Enter memory required for process %d (in Bytes)-- ",i+1);
        scanf("%d",&mp[i]);
    }
    printf("\nNo. of Blocks available in memory--%d",nob);
    printf("\n\nPROCESS\tMEMORYREQUIRED\tALLOCATED\tINTERNAL
    FRAGMENTATION");
    for(i=0;i<n && p<nob;i++)
    {
        printf("\n %d\t\t%d",i+1,mp[i]);
        if(mp[i] > bs)
            printf("\t\tNO\t\t---");
        else
        {
            printf("\t\tYES\t\t%d",bs-mp[i]);
            tif = tif + bs-mp[i];
            p++;
        }
    }
```

```

}
if(i<n)
printf("\nMemory is Full, Remaining Processes cannot be accomodated");
printf("\n\nTotal Internal Fragmentation is %d",tif);
printf("\nTotal External Fragmentation is %d",ef);
getch();
}

```

INPUT

Enter the total memory available (in Bytes) -- 1000

Enter the block size (in Bytes)-- 300

Enter the number of processes – 5

Enter memory required for process 1 (in Bytes) -- 275

Enter memory required for process 2 (in Bytes) -- 400

Enter memory required for process 3 (in Bytes) -- 290

Enter memory required for process 4 (in Bytes) -- 293

Enter memory required for process 5 (in Bytes) -- 100

No. of Blocks available in memory -- 3

OUTPUT

PROCESS ALLOCAT INTERNAL

MEMORY REQUIRED ED FRAGMENTATION

1 275 YES 25

2 400 NO -----

3 290 YES 10

4 293 YES 7

Memory is Full, Remaining Processes cannot be accommodated Total

Internal Fragmentation is 42

Total External Fragmentation is 100

B) MEMORY VARIABLE PARTIONING TYPE (MVT)

AIM: To write a program to simulate the MVT algorithm

ALGORITHM:

Step1: start the process.

Step2: Declare variables.

Step3: Enter total memory size ms.

Step4: Allocate memory for os.

$Ms = ms - os$

Step5: Read the no partition to be divided n Partition size= ms/n .

Step6: Read the process no and process size.

Step 7: If process size is less than partition size allot else block the process. While allocating update memory wastage-external fragmentation.

```
if(pn[i]==pn[j]) f=1;
```

```
if(f==0){ if(ps[i]<=size)
```

```
{
```

```
extft=extft+sizeps[i];avail[i]=1; count++;
```

```
}
```

```
}
```

Step 8: Print the results

Step 9: Stop the process.

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>

main()
{
    int ms,mp[10],i,
    temp,n=0; char ch = 'y';
    clrscr();
    printf("\nEnter the total memory available (in Bytes)-- ");
    scanf("%d",&ms);
    temp=ms;
    for(i=0;ch=='y';i++,n++)
    {
        printf("\nEnter memory required for process %d (in Bytes) -- ",i+1);
        scanf("%d",&mp[i]);
        if(mp[i]<=temp)
        {
            printf("\nMemory is allocated for Process %d ",i+1);
            temp = temp - mp[i];
        }
        else
        {
            printf("\nMemory is Full"); break;
        }
        printf("\nDo you want to continue(y/n) -- ");
        scanf(" %c", &ch);
    }
    printf("\n\nTotal Memory Available -- %d", ms);
    printf("\n\n\tPROCESS\t\tMEMORY ALLOCATED ");
    for(i=0;i<n;i++)
    printf("\n \t%d\t\t\t%d",i+1,mp[i]);
    printf("\n\nTotal Memory Allocated is %d",ms-temp);
    printf("\nTotal External Fragmentation is %d",temp);
    getch();
}
```

OUTPUT:

Enter the total memory available (in Bytes) – 1000

Enter memory required for process 1 (in Bytes) – 400

Memory is allocated for Process 1

Do you want to continue(y/n) -- y

Enter memory required for process 2 (in Bytes) -- 275

Memory is allocated for Process 2

Do you want to continue(y/n) – y

Enter memory required for process 3 (in Bytes) – 550

Memory is Full

Total Memory Available – 1000

PROCESS MEMORY ALLOCATED

1 400

2 275

Total Memory Allocated is 675

Total External Fragmentation is 325

EXECL SYSTEM CALL

AIM:

To write a program to implement the system call `execl ( )`.

ALGORITHM

Step 1: Start the program.

Step 2: Include the necessary header files.

Step 3: Print execution of `exec` system call for the `ls` Unix command.

Step 4: Execute the `execl` function using the appropriate syntax for the Unix command `ls`.

Step 5: The list of all files and directories of the system is displayed.

Step 6: Stop the program.

PROGRAM :

SOURCE CODE:

```
/* execl system call */
#include<sys/types.h>
#include<unistd.h>
#include<stdio.h>
main()
{
printf("Before execl \n");
execl("/bin/ls","ls",(char*)0);
printf("After Execl\n");
}
```

OUTPUT:

```
$ vi execl.c
```

```
$ cc execl.c
```

```
$ ./a.out
```

Before execl

a1 aaa aaa.txt abc a.out b1 b2 comm.c db db1 demo2 dir1 direc.c execl.c fl.txt fflag.c

file1 file2 fork.c m1 m2 wait.c xyz

RESULT:

Thus the program was executed and verified successfully